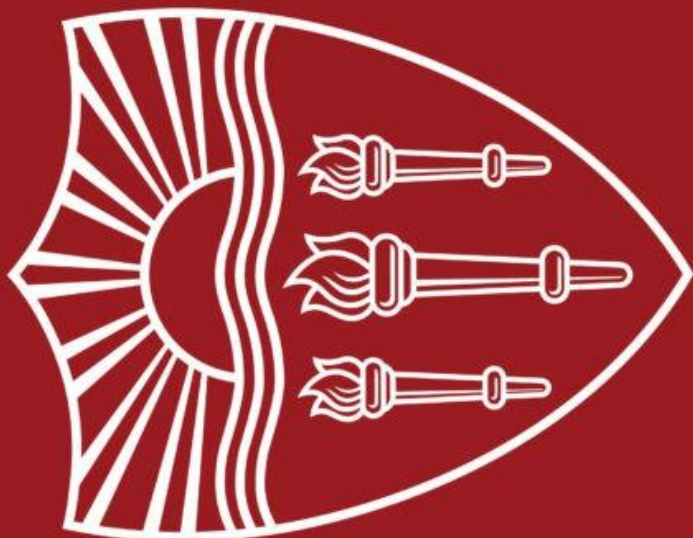


USC



Lecture 12: Transformers (contd.)

Instructor: Xiang Ren
USC CSCI 544 Applied NLP
Fall 2026

Announcements

- Proposal grades will be released by EOD – Sorry for the delay!
- CARC access pending activation – will have update by end of the week.
- Next Tue (Oct 6):
 - Guest lecture by TA on PyTorch for Transformers
- Next Thu: No class / Fall Break
- Tue 10/20: Project midterm presentation

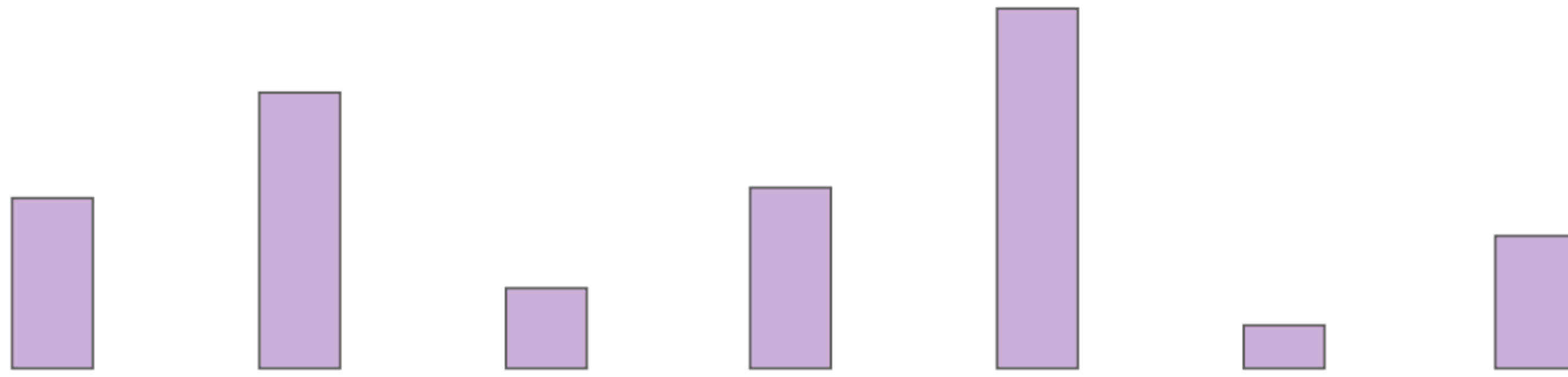
Lecture Outline

- Recap: Transformers as Encoders, Decoders and Encoder-Decoders
- The pre-training and fine-tuning paradigm
 - Pre-training Decoder-Only Models
 - Pre-training Encoder-Only Models
 - Pre-training Encoder-Decoder Models

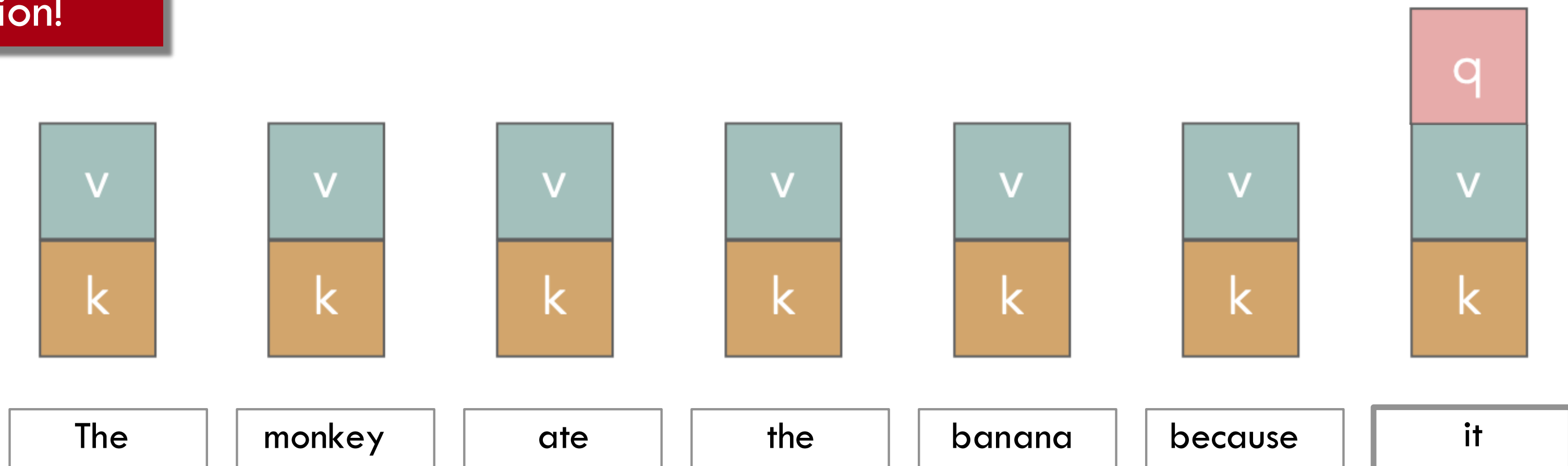
Recap: Transformers

Attention in the decoder

Attention
Distribution



Self-Attention!



Self-Attention

Keys, Queries, Values from the same sequence

- Step 1: For each word x_i , calculate its **query**, **key**, and **value**.

$$q_i = W^Q x_i \quad k_i = W^K x_i \quad v_i = W^V x_i$$

- Step 2: Calculate attention score between **query** and **keys**.

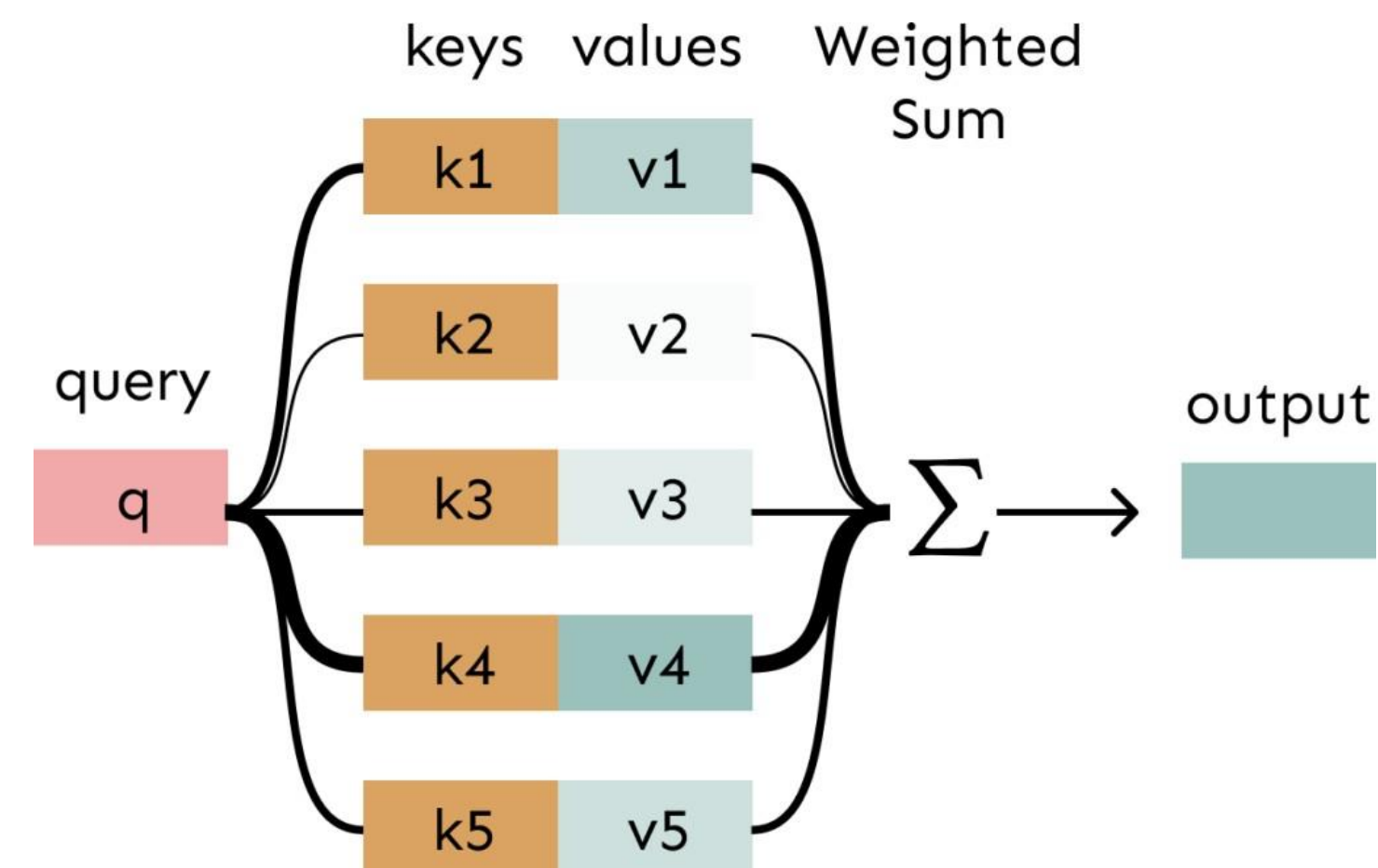
$$e_{ij} = q_i \cdot k_j$$

- Step 3: Take the softmax to normalize attention scores.

$$\alpha_{ij} = \text{softmax}(e_{ij}) = \frac{\exp(e_{ij})}{\sum_k \exp(e_{ik})}$$

- Step 4: Take a weighted sum of **values**.

$$\text{Output}_i = \sum_j \alpha_{ij} v_j$$



(Vectorized) Self-Attention in Transformer

Step 1: With embeddings stacked in X , calculate **queries**, **keys**, and **values**.

$$Q = XW^Q \quad K = XW^K \quad V = XW^V$$

Step 2: Calculate attention scores between **query** and **keys**.

$$E = QK^T$$

Step 3: Take the softmax to normalize attention scores.

$$A = \text{softmax}(E)$$

Step 4: Take a weighted sum of **values**.

$$\text{Output} = AV$$

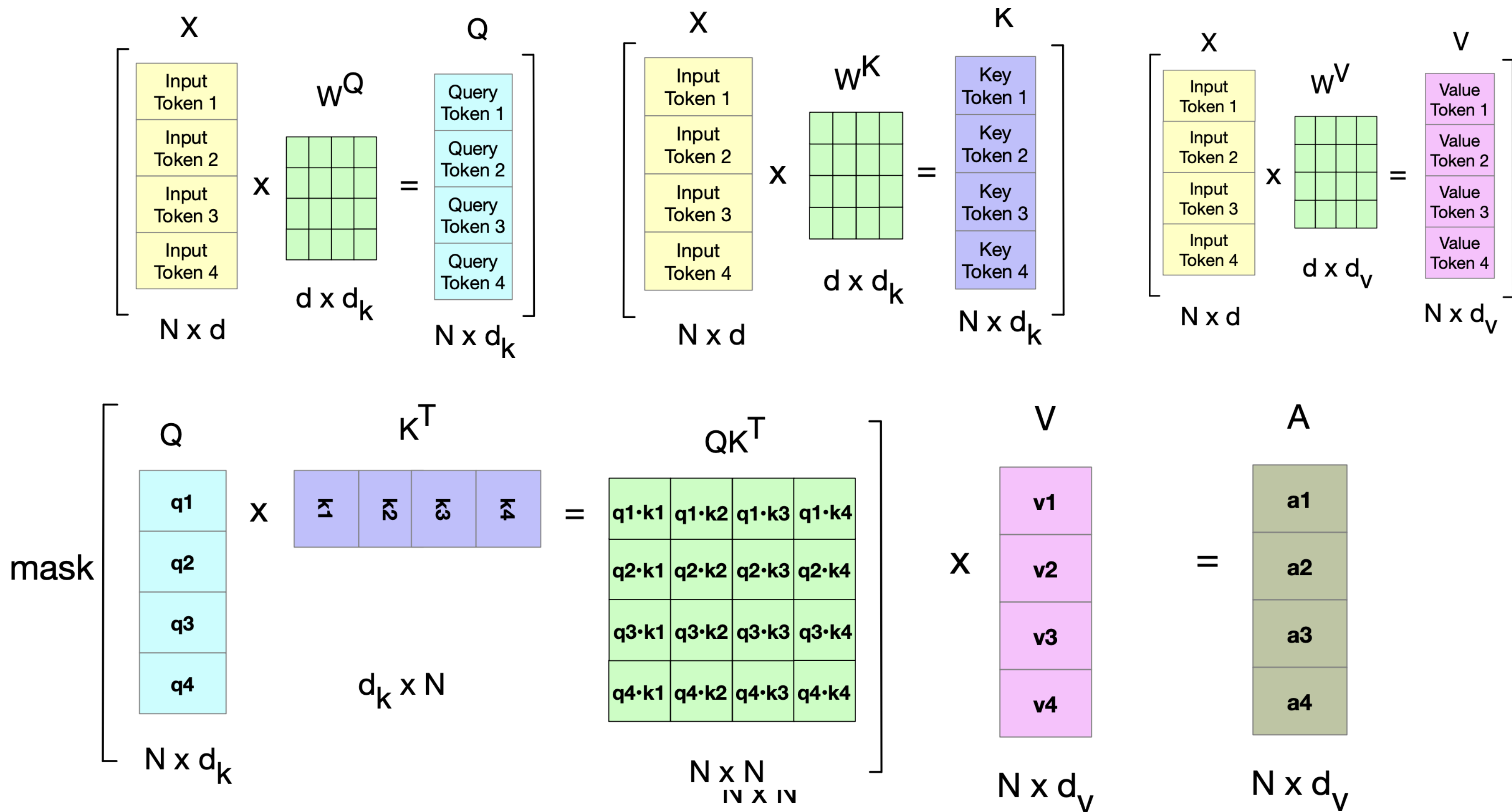
$$\text{Output} = \text{softmax}(QK^T) V$$

$$QK^T$$

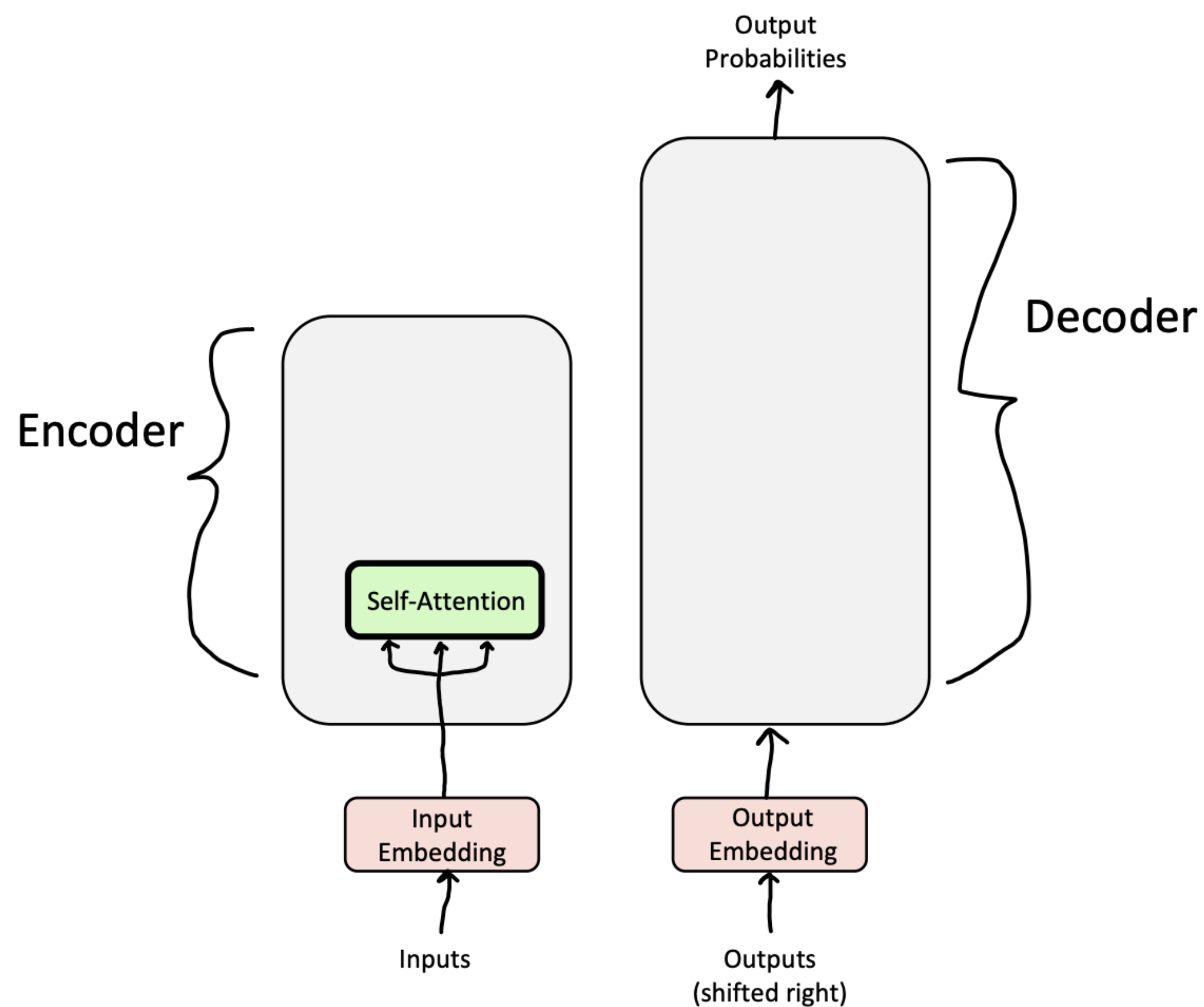
Now can do a single matrix multiply to combine Q and K^T

N	q1•k1	q1•k2	q1•k3	q1•k4
	q2•k1	q2•k2	q2•k3	q2•k4
	q3•k1	q3•k2	q3•k3	q3•k4
	q4•k1	q4•k2	q4•k3	q4•k4
N				

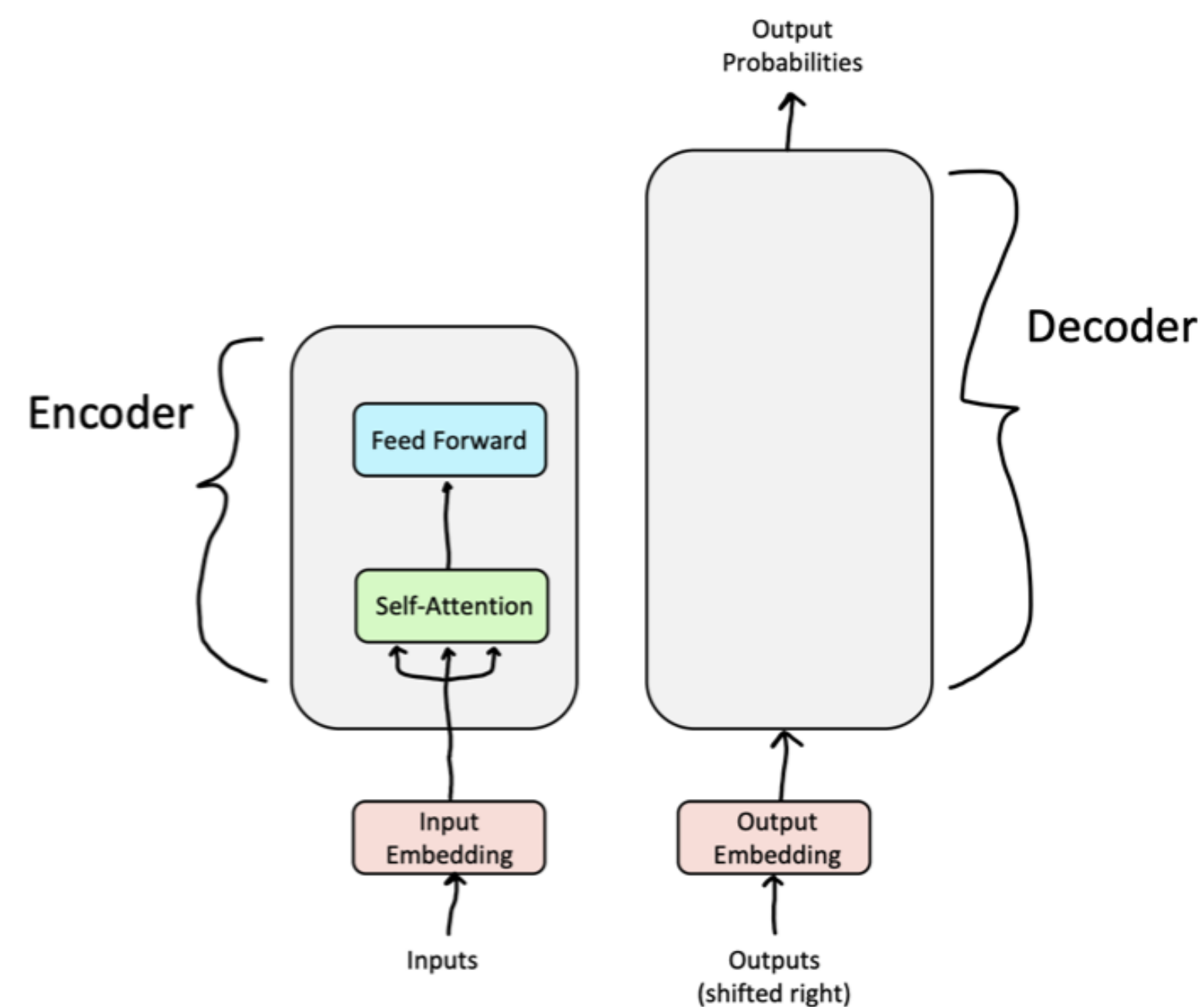
Attention Overall



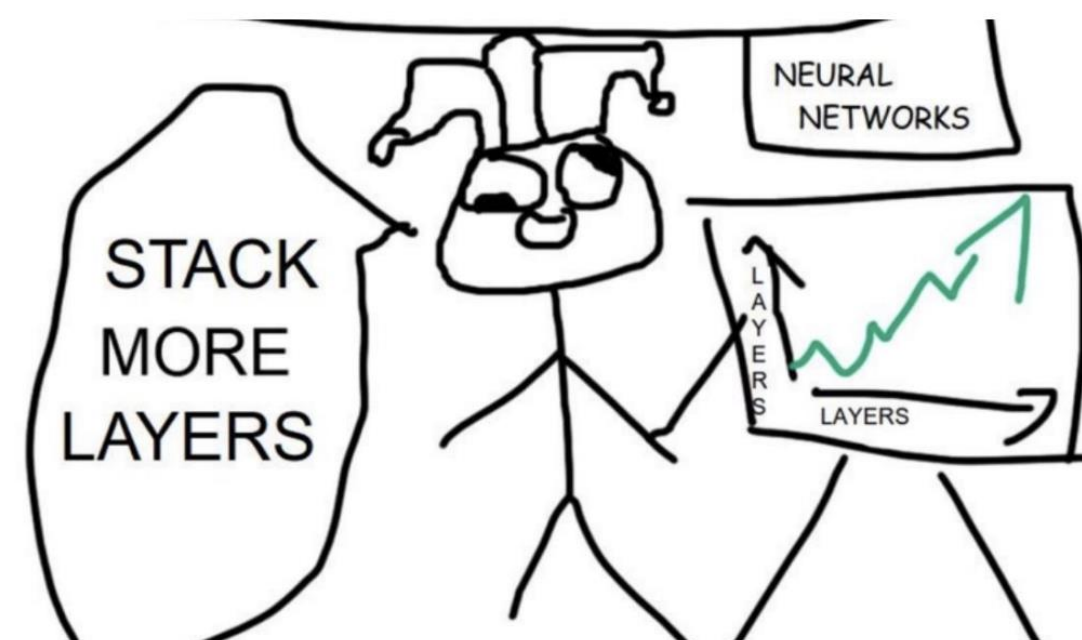
What We Have So Far: (Encoder) Self-Attention!



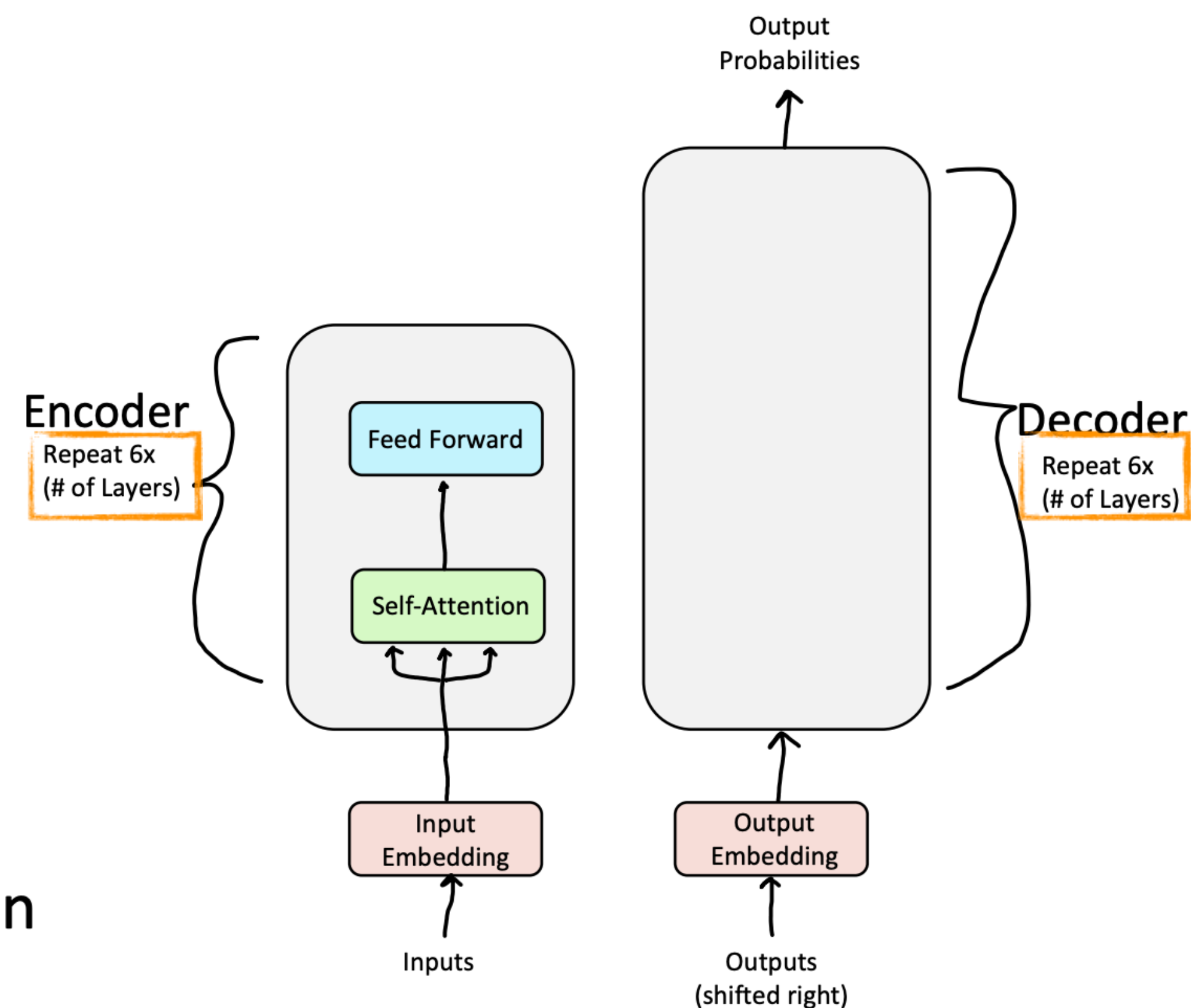
But Not Enough!



How do we make this work for **deep** NNs?



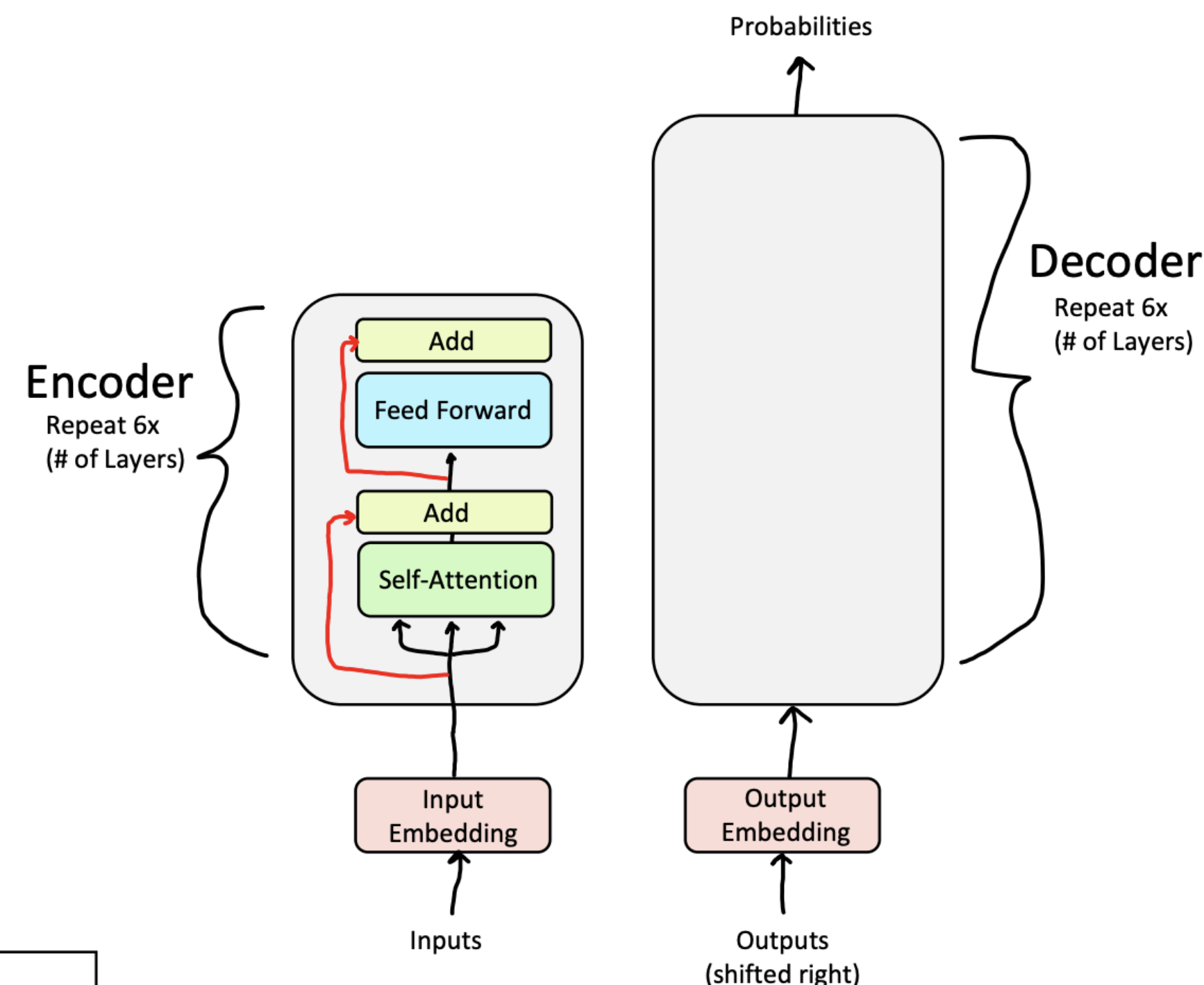
- Training Trick #1: Residual Connections
- Training Trick #2: LayerNorm
- Training Trick #3: Scaled Dot Product Attention



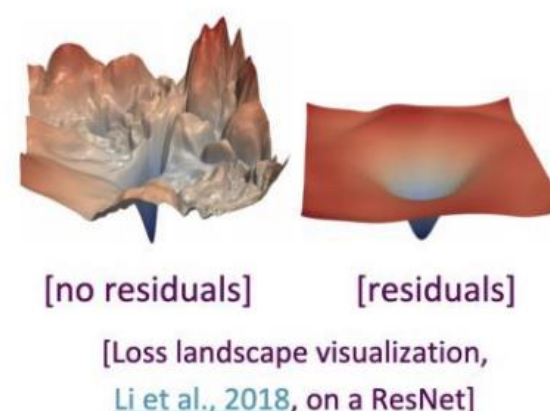
Training Trick #1: Residual Connections [He et al., 2016]

- Residual connections are a simple but powerful technique from computer vision.
- Deep networks are surprisingly bad at learning the identity function!
- Therefore, directly passing "raw" embeddings to the next layer can actually be very helpful!

$$x_\ell = F(x_{\ell-1}) + x_{\ell-1}$$
- This prevents the network from "forgetting" or distorting important information as it is processed by many layers.



Residual connections are also thought to smooth the loss landscape and make training easier!

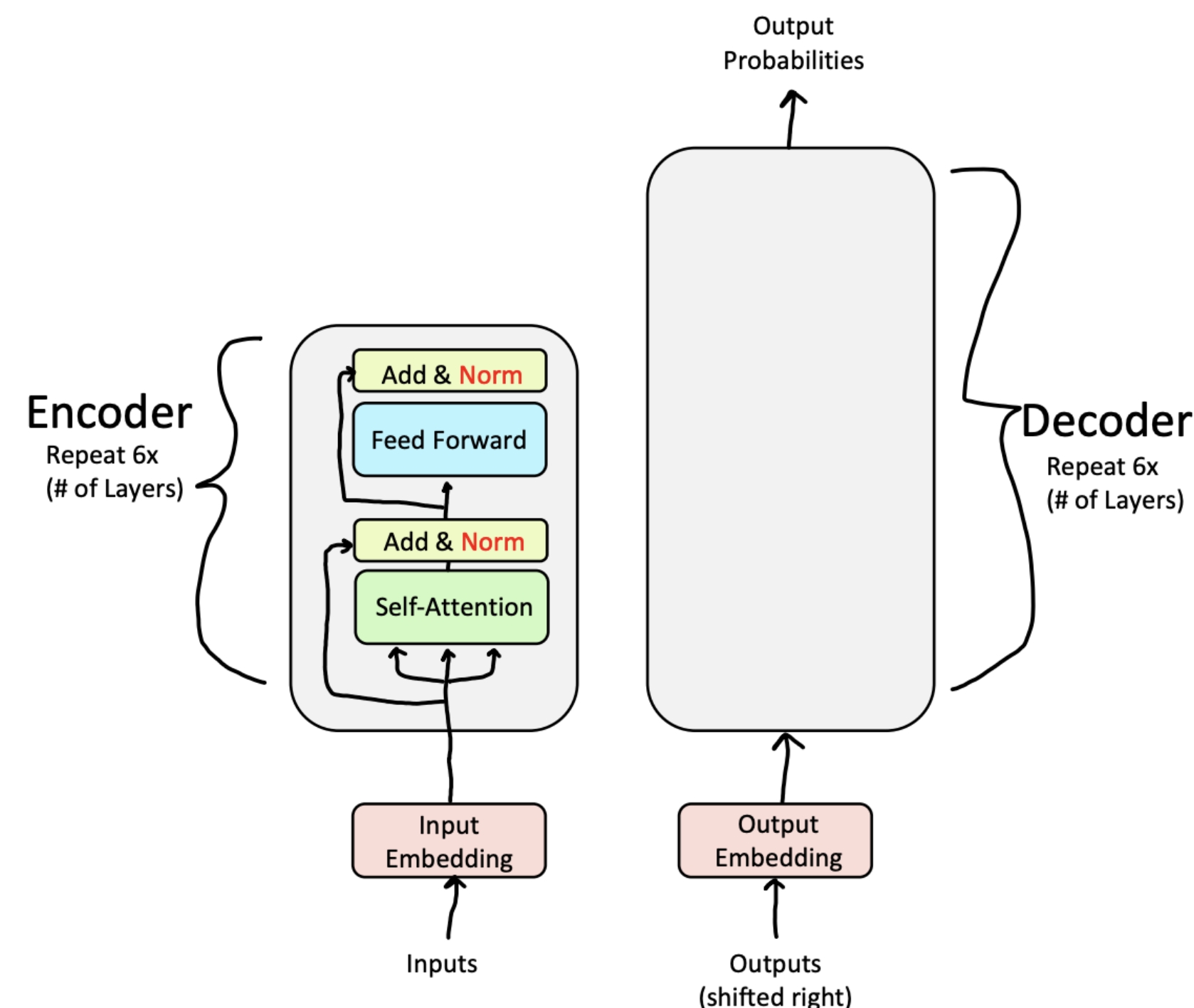


Training Trick #2: Layer Normalization [Ba et al., 2016]

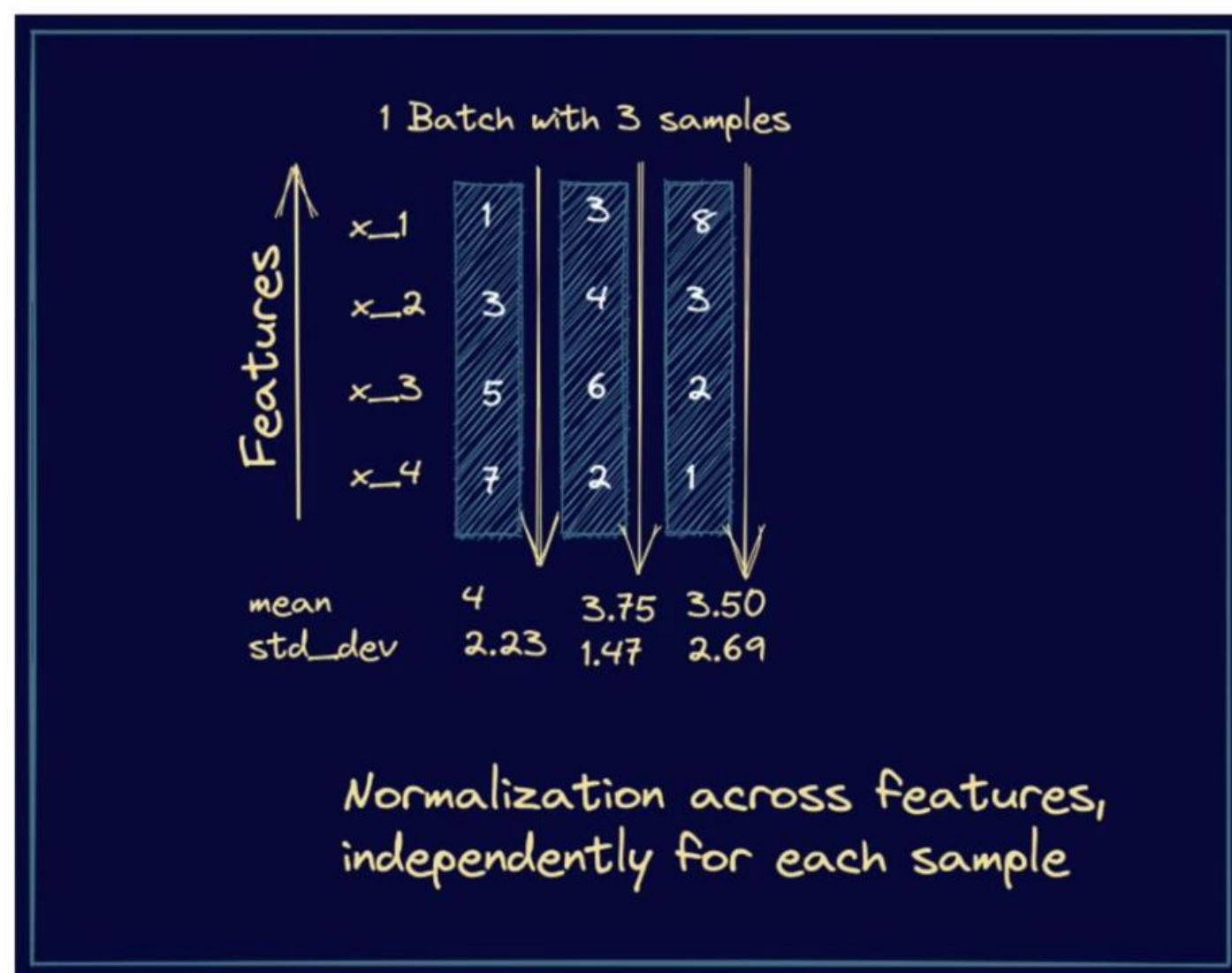
- **Problem:** Difficult to train the parameters of a given layer because its input from the layer beneath keeps shifting.
- **Solution:** Reduce variation by **normalizing** to zero mean and standard deviation of one within each **layer**.

$$\text{Mean: } \mu^l = \frac{1}{H} \sum_{i=1}^H a_i^l \quad \text{Standard Deviation: } \sigma^l = \sqrt{\frac{1}{H} \sum_{i=1}^H (a_i^l - \mu^l)^2}$$

$$x^{\ell'} = \frac{x^{\ell} - \mu^{\ell}}{\sigma^{\ell} + \epsilon}$$



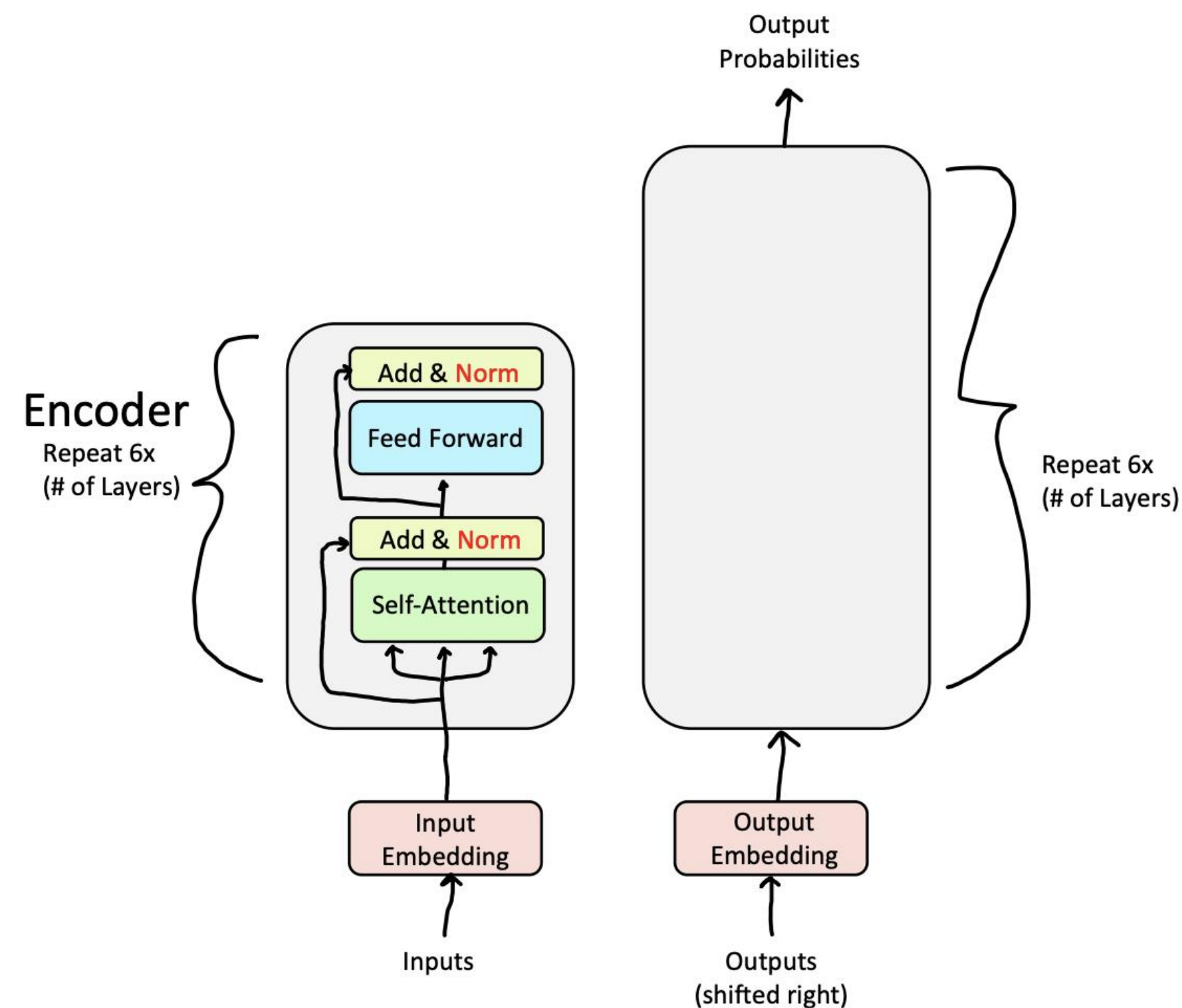
Training Trick #2: Layer Normalization [Ba et al., 2016]



An Example of How LayerNorm Works (Image by Bala Priya C, Pinecone)

Mean: $\mu^l = \frac{1}{H} \sum_{i=1}^H a_i^l$ Standard Deviation: $\sigma^l = \sqrt{\frac{1}{H} \sum_{i=1}^H (a_i^l - \mu^l)^2}$

$$x^{\ell'} = \frac{x^{\ell} - \mu^{\ell}}{\sigma^{\ell} + \epsilon}$$



Layer Normalization

- A trick to help models train faster
- Idea: cut down on uninformative variation in hidden vector values by normalizing to unit mean and standard deviation within each layer
- Let $x \in \mathbb{R}^d$ be an individual (word) vector in the model.

$$\mu = \frac{1}{d} \sum_{j=1}^d x_j; \quad \mu \in \mathbb{R}$$

$$\sigma = \sqrt{\frac{1}{d} \sum_{j=1}^d (x_j - \mu)^2}; \quad \sigma \in \mathbb{R}$$

Result: New vector with zero mean and a standard deviation of one

$$\hat{x} = \frac{x - \mu}{\sigma}$$

Component-wise subtraction

- Let $\gamma \in \mathbb{R}$ and $\beta \in \mathbb{R}^d$ be learned “gain” and “bias” parameters. (Can omit!)

$$\text{LayerNorm} = \gamma \hat{x} + \beta$$

Training Trick #3: Scaled Dot Product Attention

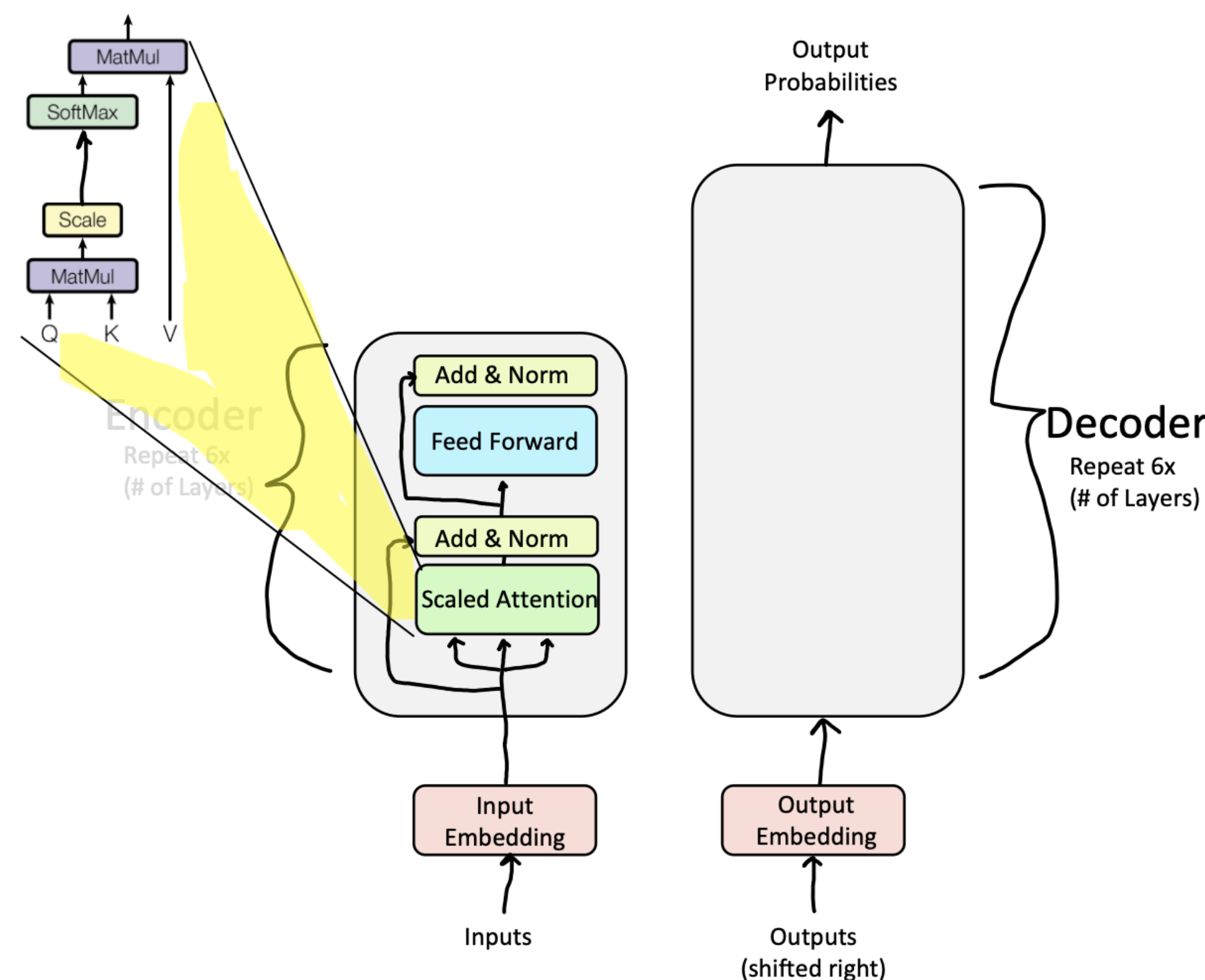
- After LayerNorm, the mean and variance of vector elements is 0 and 1, respectively. (Yay!)
- However, the dot product still tends to take on extreme values, as its variance scales with dimensionality d_k

Quick Statistics Review:

- Mean of sum = sum of means = $d_k * 0 = 0$
- Variance of sum = sum of variances = $d_k * 1 = d_k$
- To set the variance to 1, simply divide by $\sqrt{d_k}$!

Updated Self-Attention Equation:

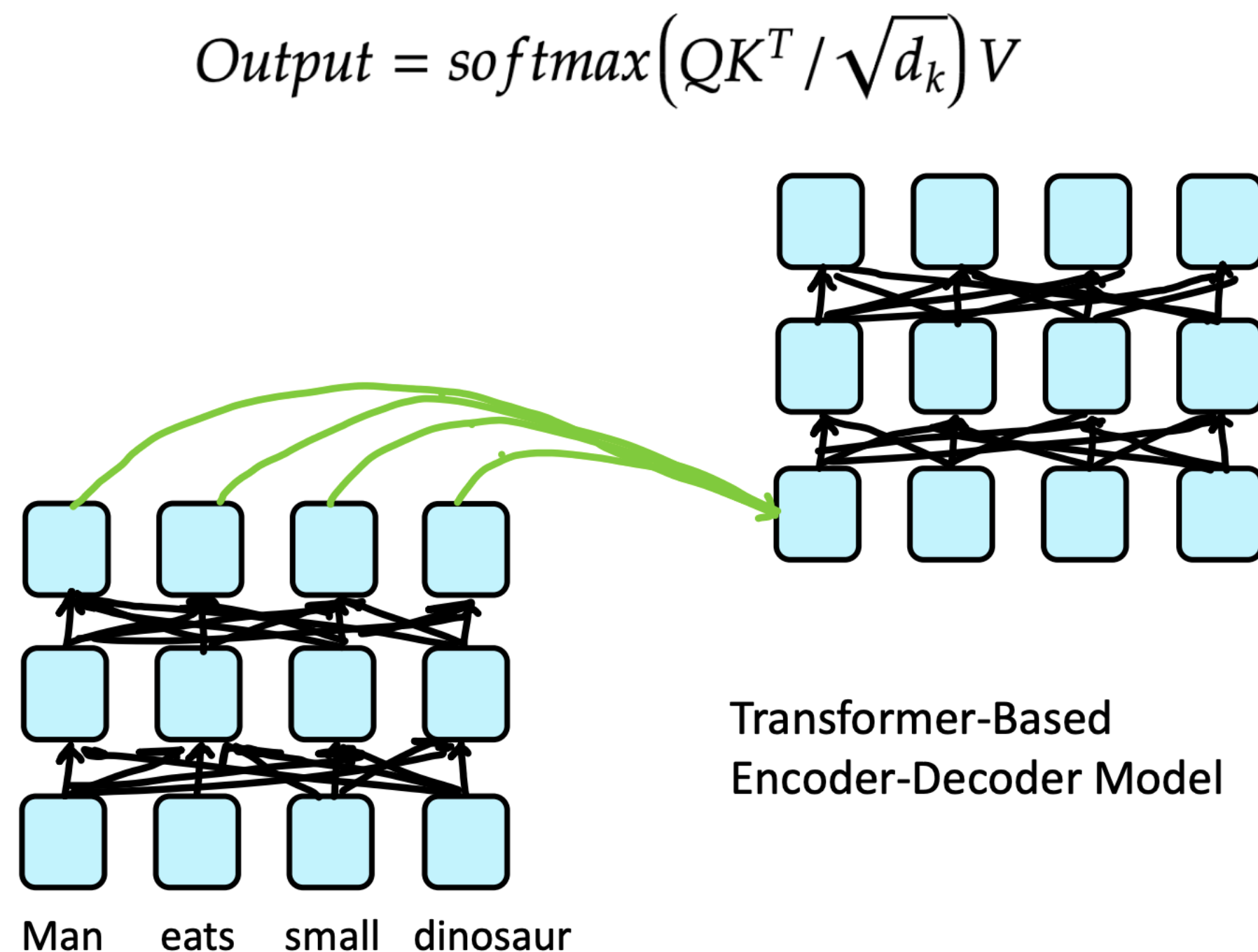
$$Output = softmax(QK^T / \sqrt{d_k}) V$$



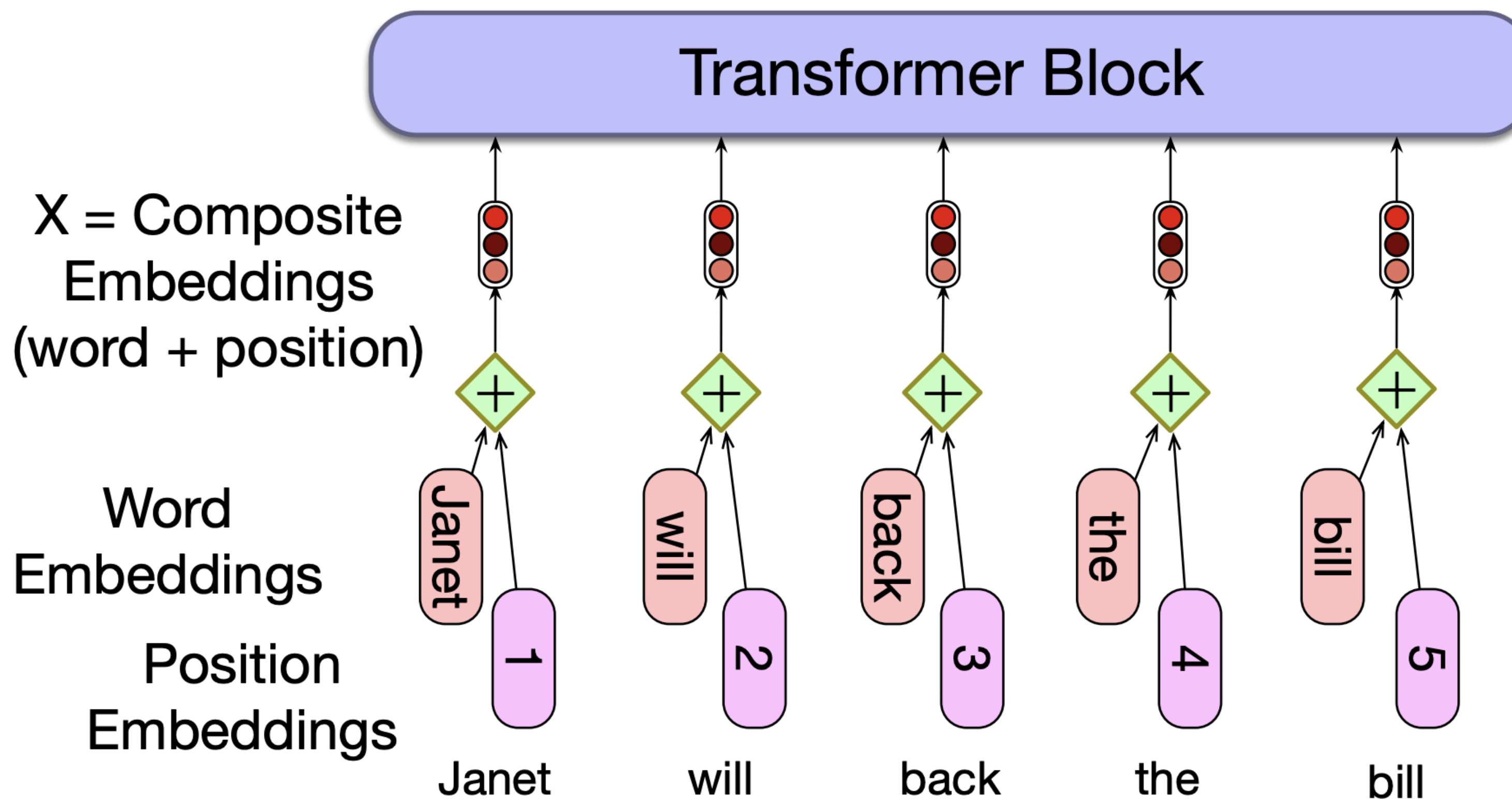
Major Issue

We are almost done with Encoder, but we have one major problem!

- Consider the sentence:
 - “Man eats small dinosaur.”
- Wait a minute, order doesn't impact the network at all!
- This seems wrong given that word order does have meaning in many languages, e.g, in EN.



Sequence Order



Solution: Position embedding

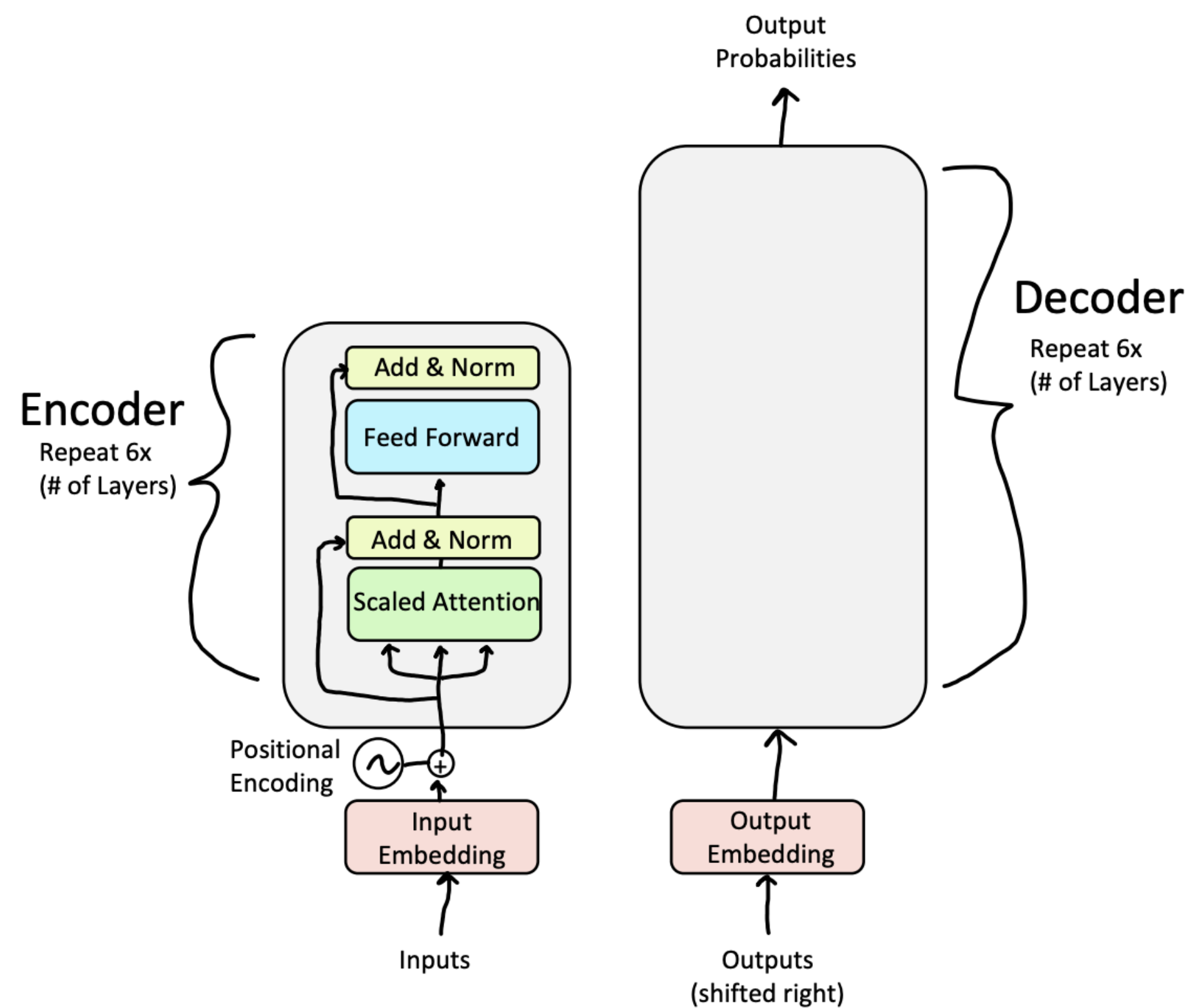
- Consider representing each sequence index as a vector
 - $\mathbf{p}_i \in \mathbb{R}^d$, for $i \in \{1, 2, \dots, n\}$ are position vectors
- Don't worry about what the $\mathbf{p}_i$ are made of yet!
- Easy to incorporate this info: just add the $\mathbf{p}_i$ to our inputs!
- Recall that $\mathbf{x}_i$ is the embedding of the word at index i . The positioned embedding is:

$$\tilde{\mathbf{x}}_i = \mathbf{x}_i + \mathbf{p}_i$$

In deep self-attention networks, we do this at the first layer! You could concatenate them as well, but people mostly just add...

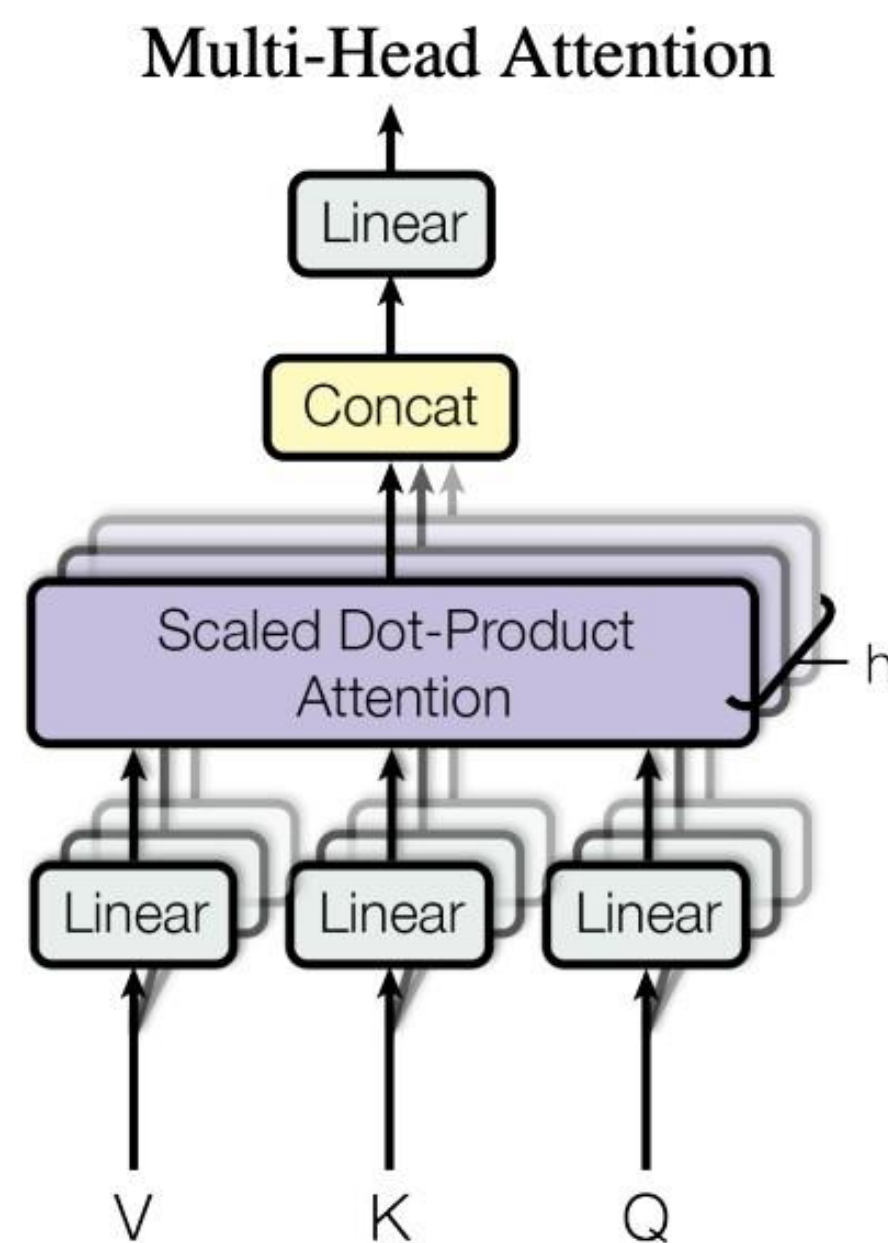
Solution

Inject order information through Positional Embeddings!



Multi-Headed Self-Attention

High Level Idea: perform self-attention multiple times in parallel and combine the results



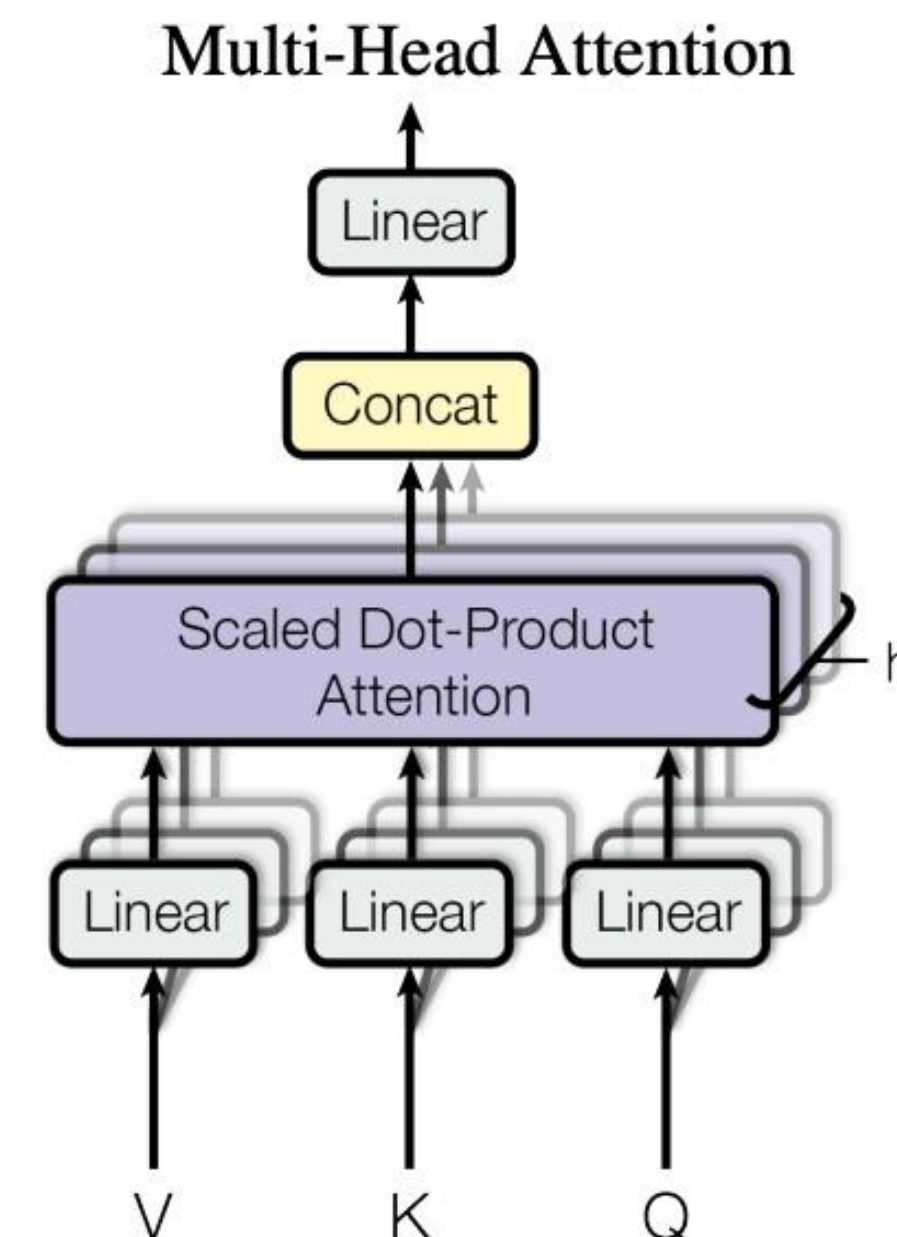
Wizards of the Coast, Artist: Todd Lockwood

Multi-headed attention

- What if we want to look in multiple places in the sentence at once?
 - For word i , self-attention “looks” where $\mathbf{x}_i^T \mathbf{Q}^T (\mathbf{K} \mathbf{x}_j)$ is high, but maybe we want to focus on different j for different reasons?
- Define multiple attention “heads” through multiple $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ matrices
- Let $\mathbf{Q}_l, \mathbf{K}_l, \mathbf{V}_l$, each in $\mathbb{R}^{d \times \frac{d}{h}}$, where h is the number of attention heads, and $1 \leq l \leq h$.
- Each attention head performs attention independently:

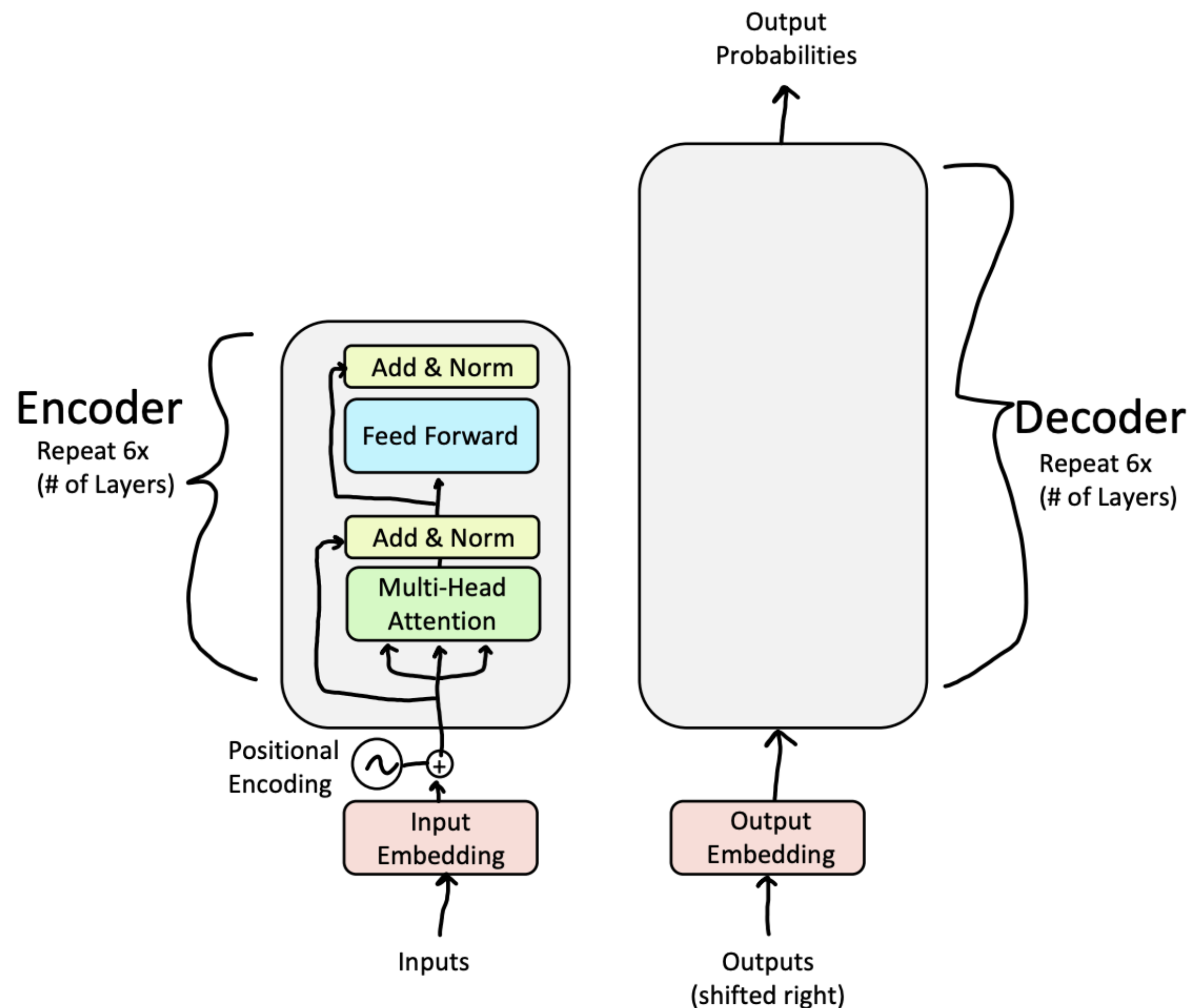
$$\text{output}_\ell = \underset{\mathbb{R}^{d/h}}{\text{softmax}}(X Q_\ell K_\ell^T X^T) * X V_\ell, \text{ where } \text{output}_\ell \in \mathbb{R}^{d/h}$$

- Then the outputs of all the heads are combined!



Each head gets to “look” at different things, and construct value vectors differently

Yay, we've completed the Encoder! Time for Decoder ...



Decoder: Masked Multi-Head Self-Attention

- Problem: How do we keep the decoder from “cheating” — looking into the future
- Solution: Masked Multi-Head Attention.
- To enable parallelization, we mask out attention to future words by setting attention scores to $-\infty$

$$e_{ij} = \begin{cases} q_i^\top k_j, & j < i \\ -\infty, & j \geq i \end{cases}$$

For encoding these words

We can look at these (not greyed out) words

	[START]	The	chef	who
[START]		$-\infty$	$-\infty$	$-\infty$
The			$-\infty$	$-\infty$
chef				$-\infty$
who				

Masking out the future

$$\mathbf{A} = \text{softmax} \left(\text{mask} \left(\frac{\mathbf{QK}^\top}{\sqrt{d_k}} \right) \right) \mathbf{V}$$

Add $-\infty$ to cells in upper triangle

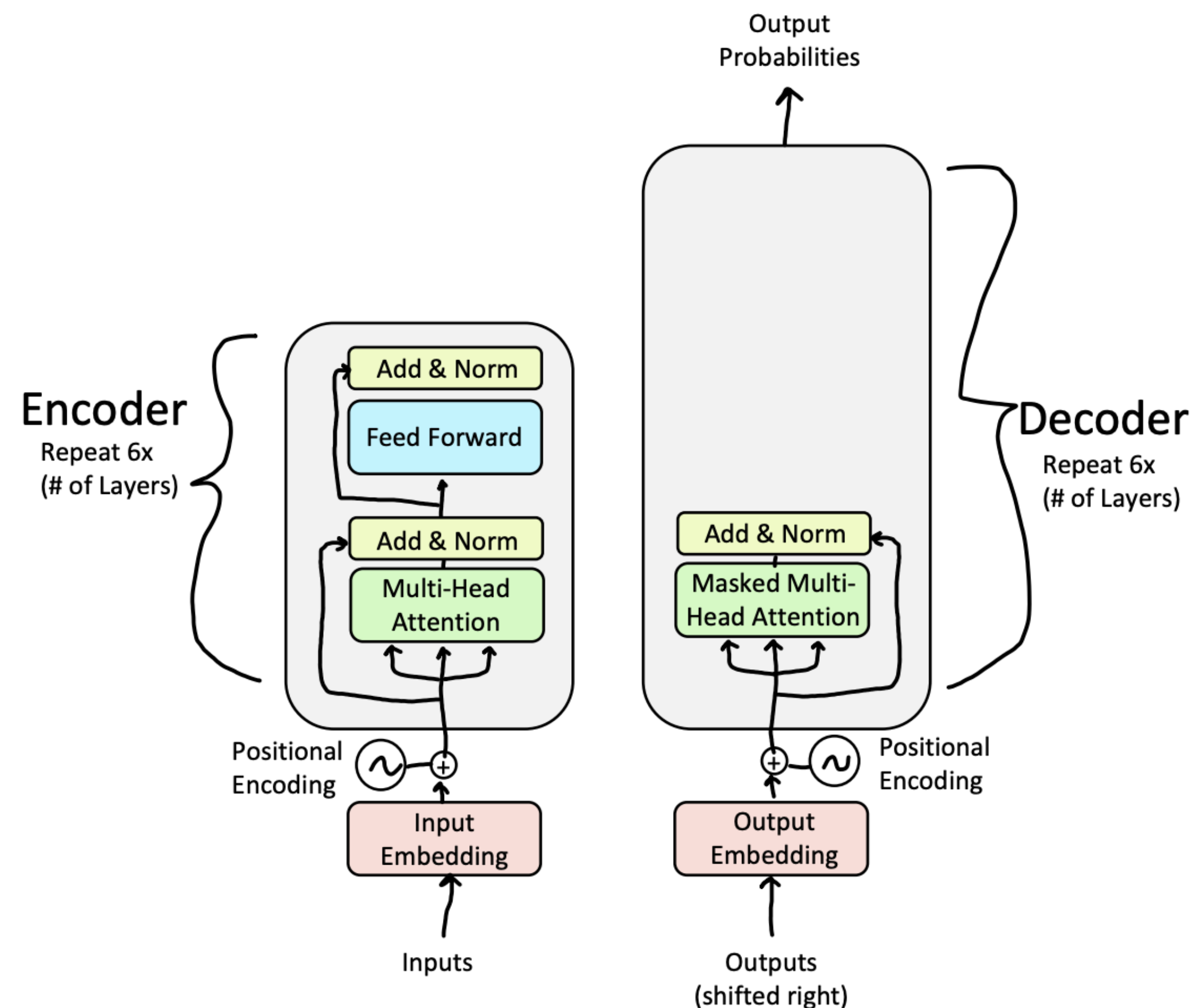
The softmax will turn it to 0

N

q1•k1	$-\infty$	$-\infty$	$-\infty$
q2•k1	q2•k2	$-\infty$	$-\infty$
q3•k1	q3•k2	q3•k3	$-\infty$
q4•k1	q4•k2	q4•k3	q4•k4

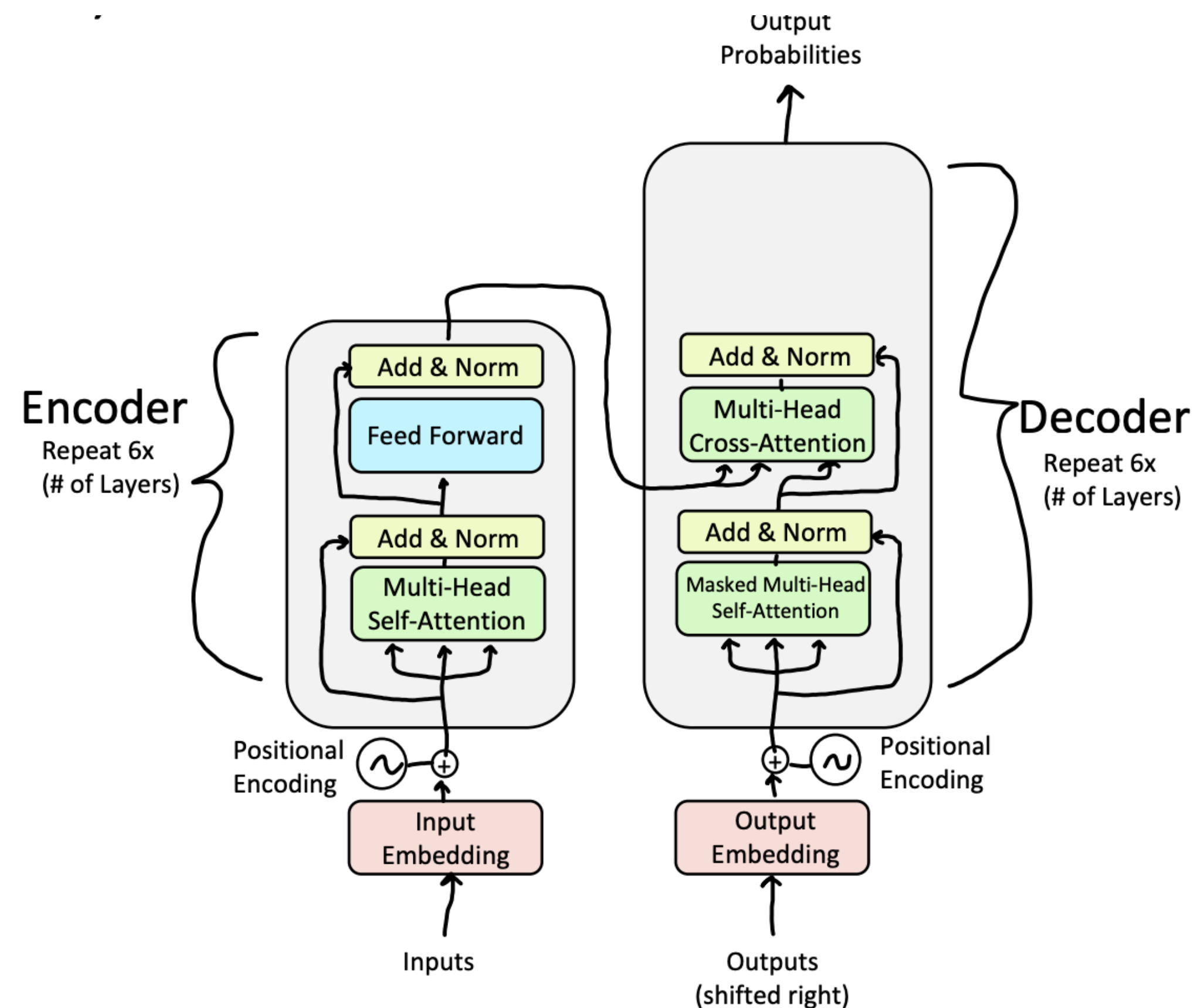
N

Decoder: Masked Multi-Head Self-Attention



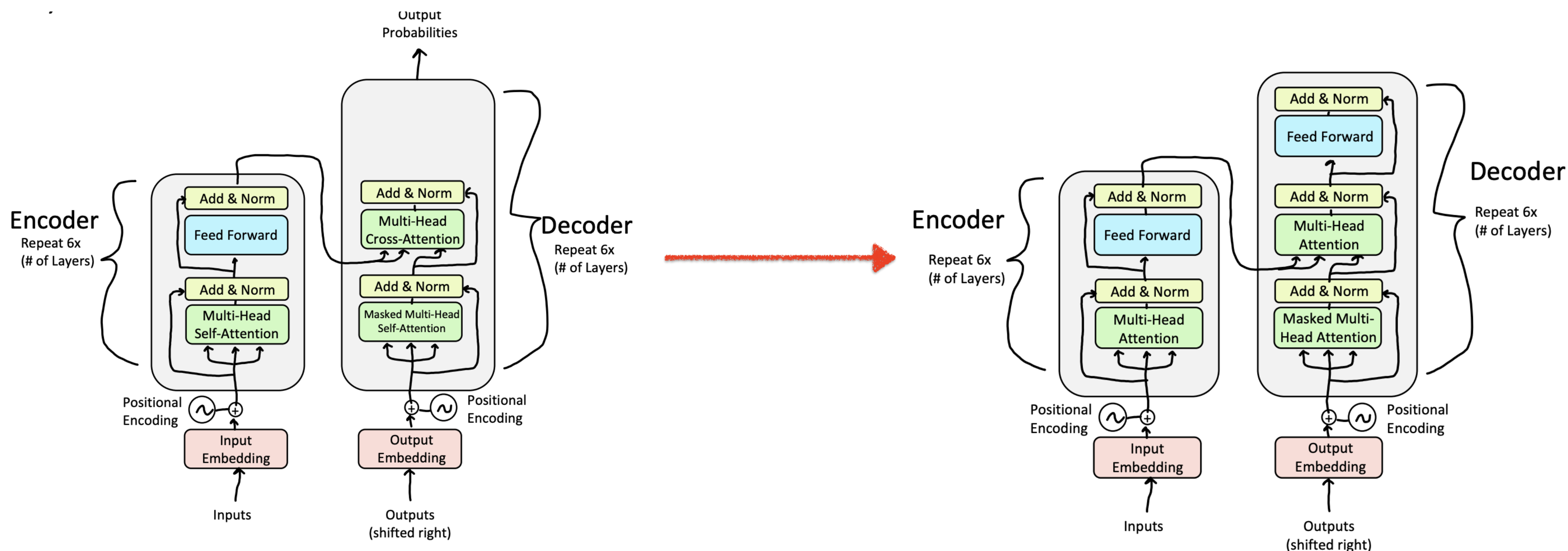
Encoder-Decoder Attention (Cross Attention)

- We saw that self -attention is when keys, queries, and values come from the same source.
- In the decoder, we have attention that looks more like what we saw last week.
- Let $\mathbf{h}_1, \dots, \mathbf{h}_n$ be output vectors from the Transformer encoder, $\mathbf{h}_i \in \mathbb{R}^d$
- Let $\mathbf{z}_1, \dots, \mathbf{z}_n$ be input vectors from the Transformer decoder, $\mathbf{z}_i \in \mathbb{R}^d$
- Then keys and values are drawn from the encoder (like a memory):
 - $\mathbf{k}_i = \mathbf{K}\mathbf{h}_i$, $\mathbf{v}_i = \mathbf{V}\mathbf{h}_i$
- And the queries are drawn from the decoder, $\mathbf{q}_i = \mathbf{Q}\mathbf{z}_i$



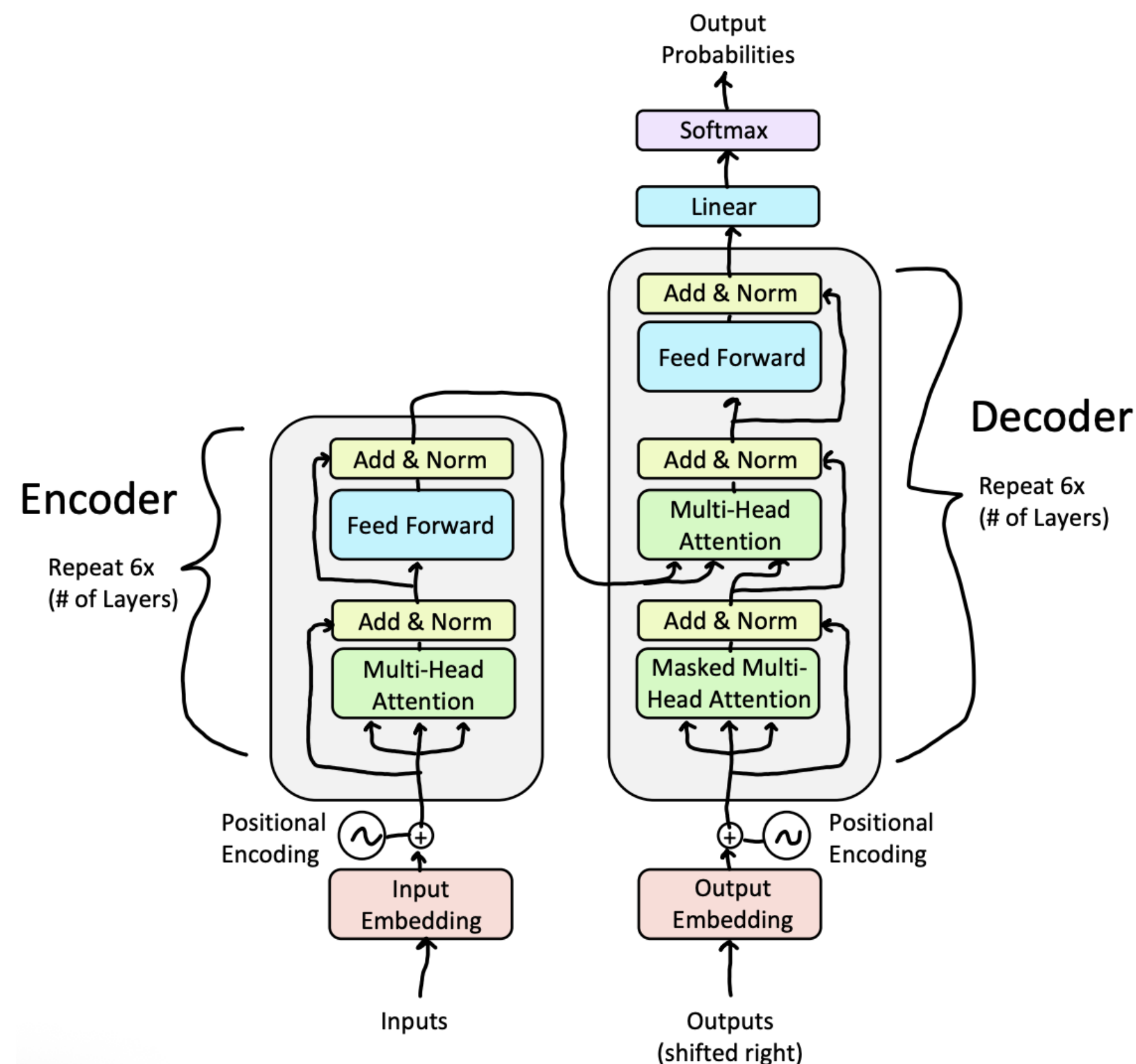
Decoder: Finishing touches!

- Add a feedforward layer (w/ residual connection and layer norm)



Decoder: Finishing touches!

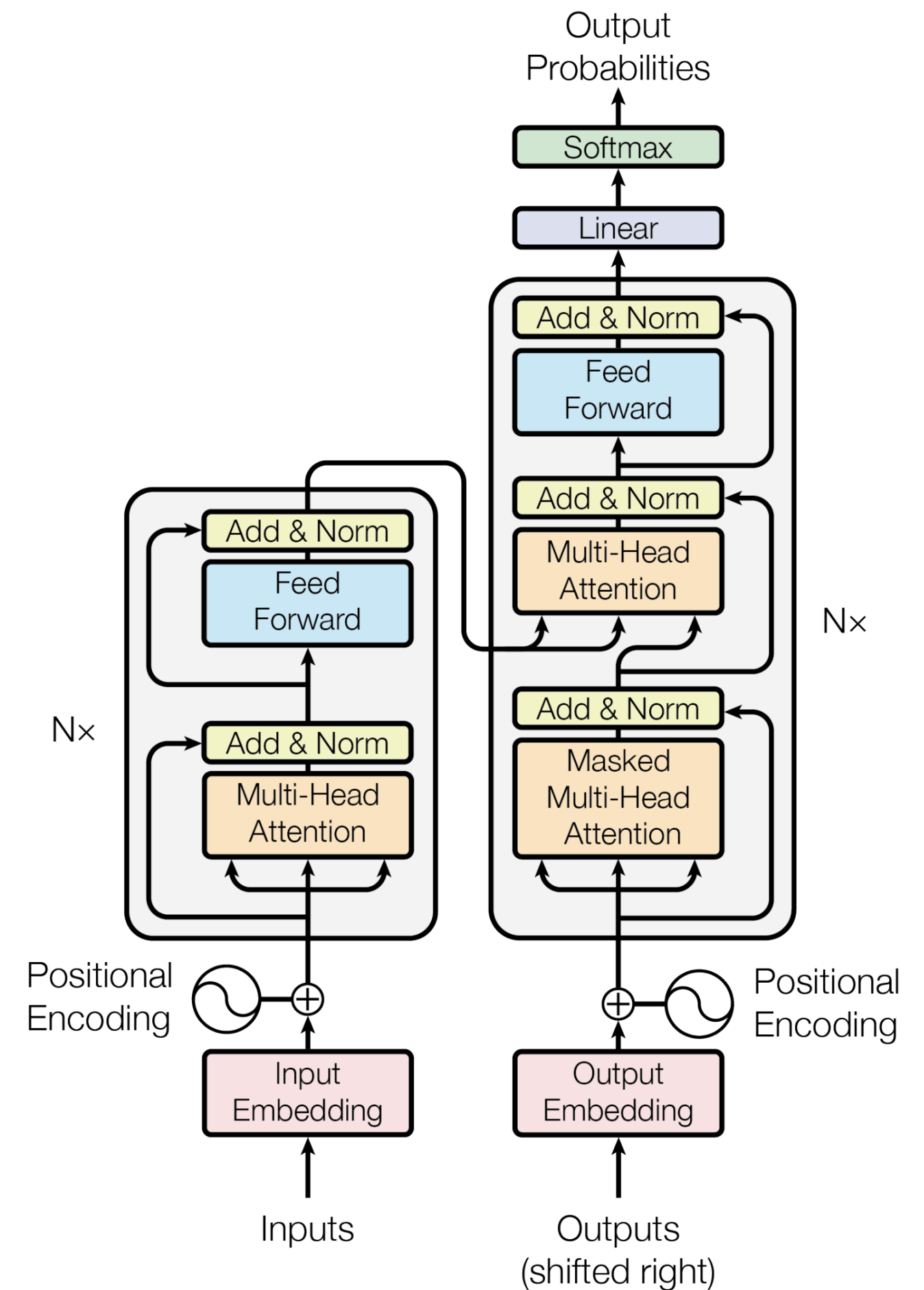
- Add a final **linear layer** to project the embeddings into a much longer vector of length vocab size (logits)
- Add a final **softmax** to generate a probability distribution of possible next words!



Transformers Have Revolutionized NLP

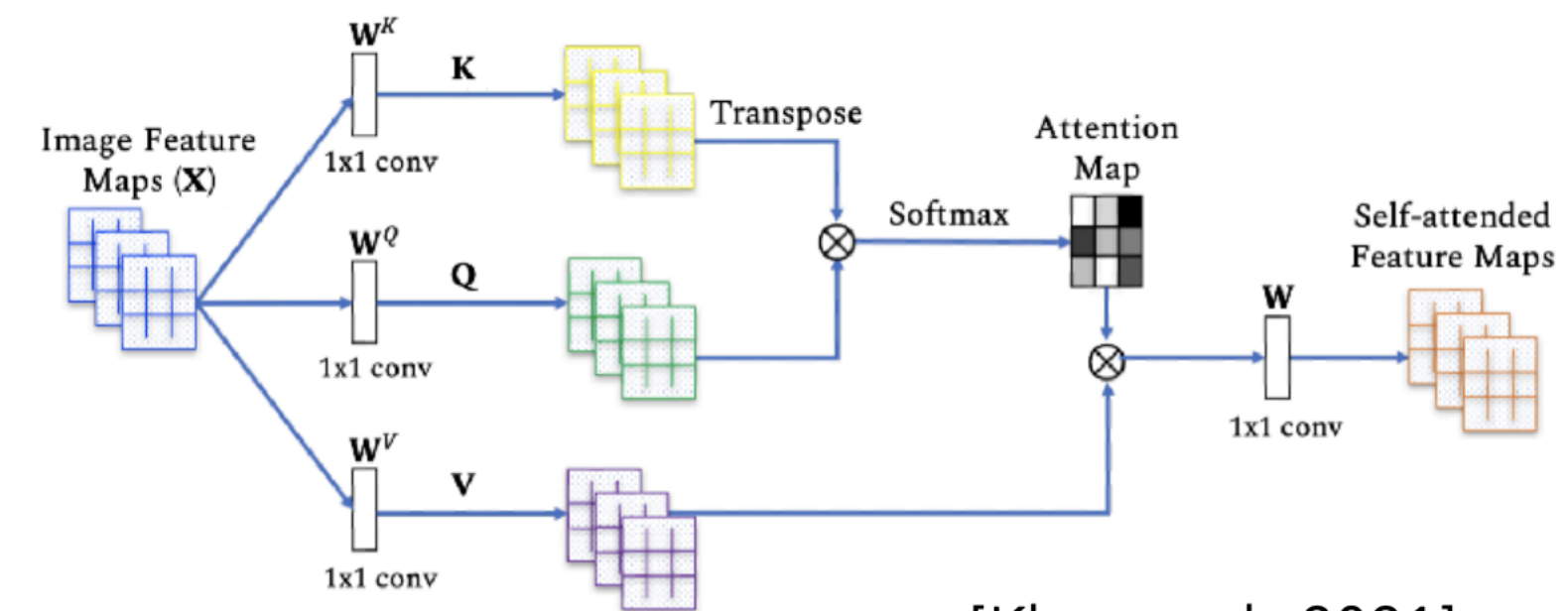


Courtesy of Paramount Pictures



The Revolutionary Impact of Transformers

- **Almost all current-day leading language models** use Transformer building blocks.
 - E.g., GPT1/2/3/4, T5, Llama 1/2, BERT, ... almost anything we can name
 - Transformer-based models dominate nearly all NLP leaderboards.
- Since Transformer has been popularized in language applications, computer vision also adapted Transformers, e.g., **Vision Transformers**.

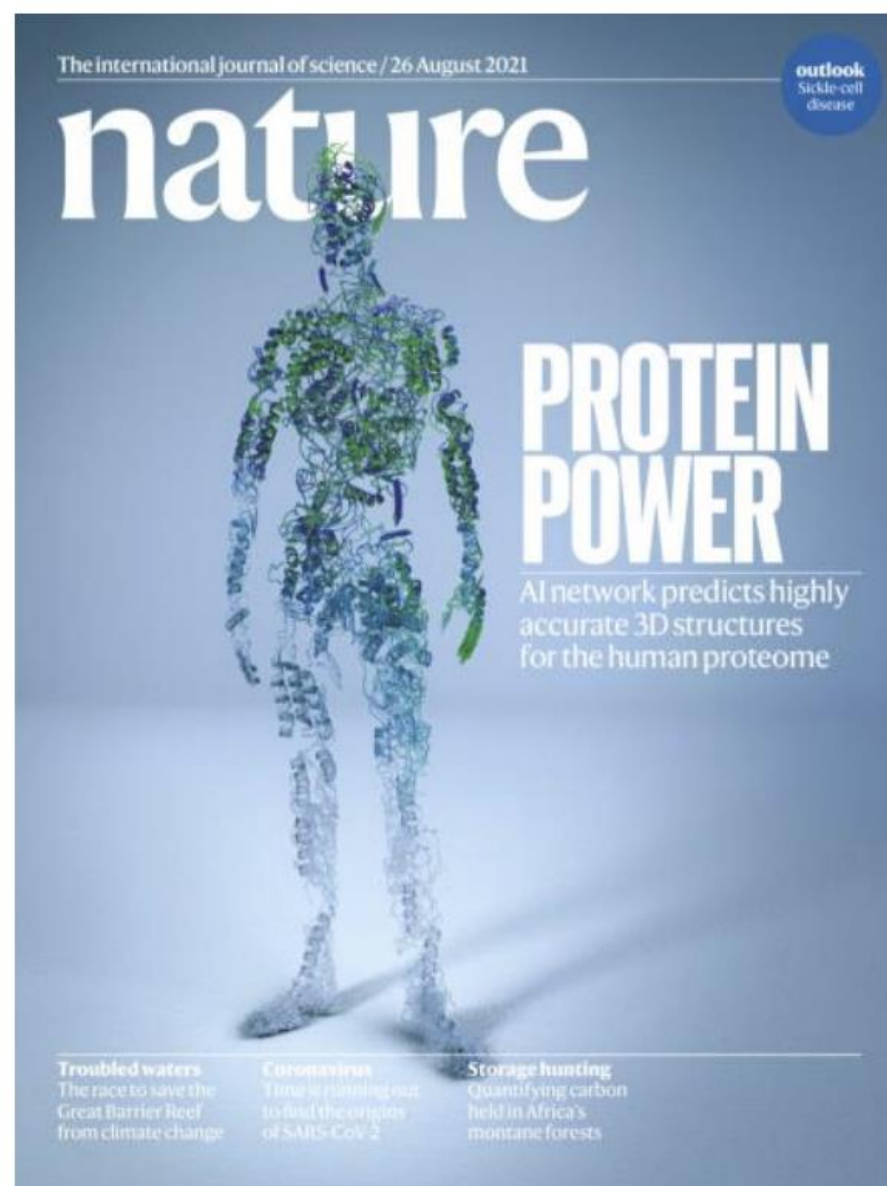


[Khan et al., 2021]

What's next after
Transformers?

Transformers Even Show Promise Outside NLP

Protein Folding



[Jumper et al. 2021] aka AlphaFold2!

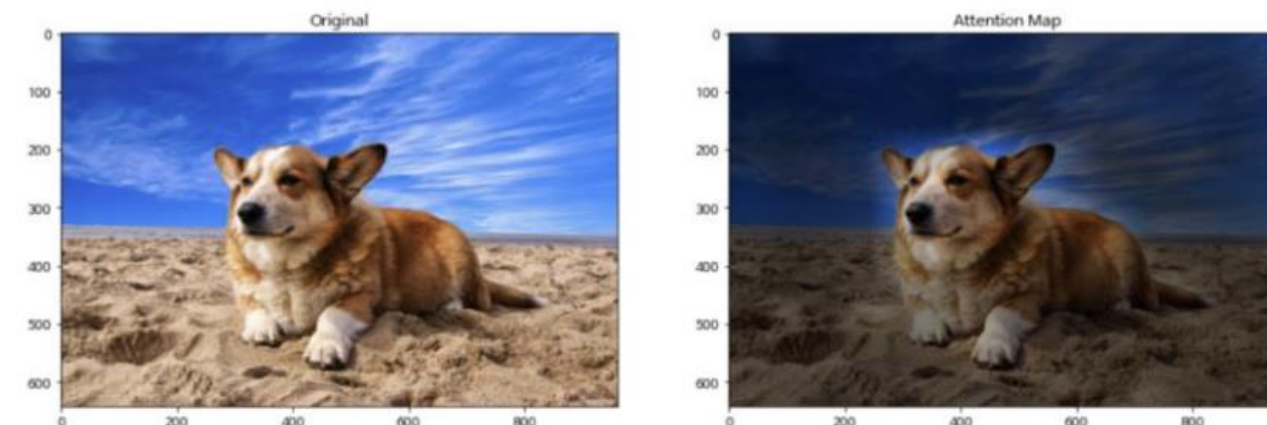
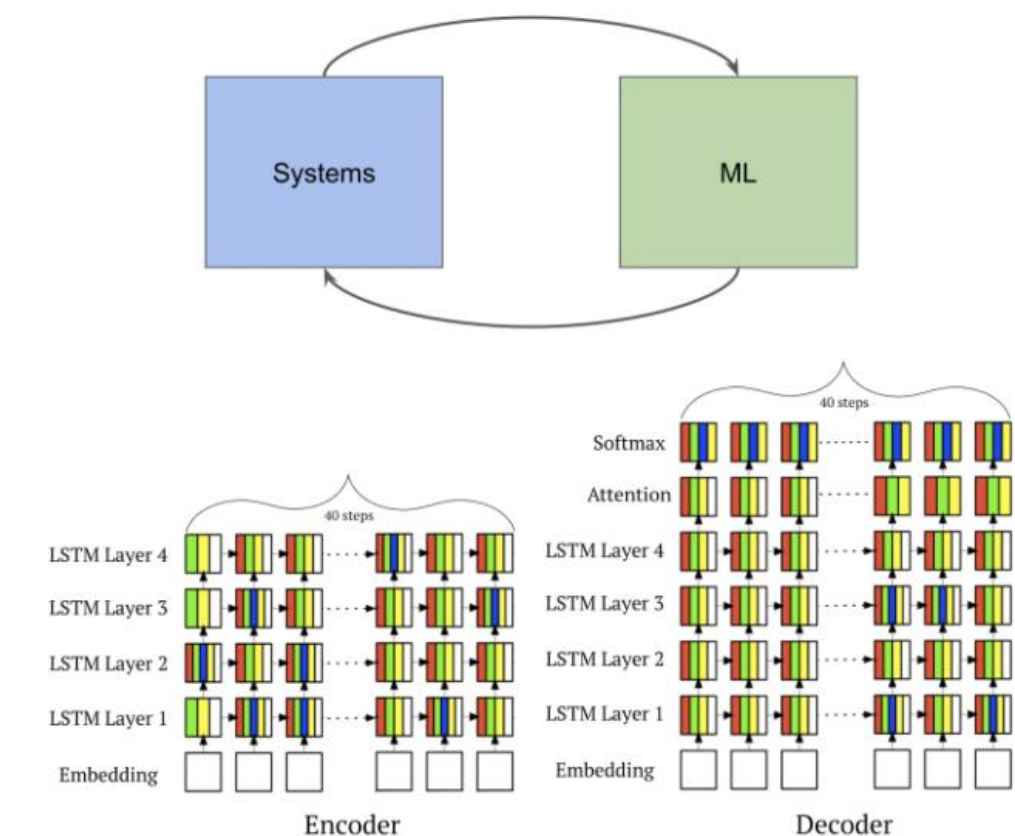


Image Classification

[Dosovitskiy et al. 2020]: Vision Transformer (ViT) outperforms ResNet-based baselines with substantially less compute.

	Ours-JFT (ViT-H/14)	Ours-JFT (ViT-L/16)	Ours-I21k (ViT-L/16)	BiT-L (ResNet152x4)	Noisy Student (EfficientNet-L2)
ImageNet	88.55 ± 0.04	87.76 ± 0.03	85.30 ± 0.02	87.54 ± 0.02	88.4/88.5*
ImageNet ReaL	90.72 ± 0.05	90.54 ± 0.03	88.62 ± 0.05	90.54	90.55
CIFAR-10	99.50 ± 0.06	99.42 ± 0.03	99.15 ± 0.03	99.37 ± 0.06	—
CIFAR-100	94.55 ± 0.04	93.90 ± 0.05	93.25 ± 0.05	93.51 ± 0.08	—
Oxford-IIIT Pets	97.56 ± 0.03	97.32 ± 0.11	94.67 ± 0.15	96.62 ± 0.23	—
Oxford Flowers-102	99.68 ± 0.02	99.74 ± 0.00	99.61 ± 0.02	99.63 ± 0.03	—
VTAB (19 tasks)	77.63 ± 0.23	76.28 ± 0.46	72.72 ± 0.21	76.29 ± 1.70	—
TPUv3-core-days	2.5k	0.68k	0.23k	9.9k	12.3k



ML for Systems

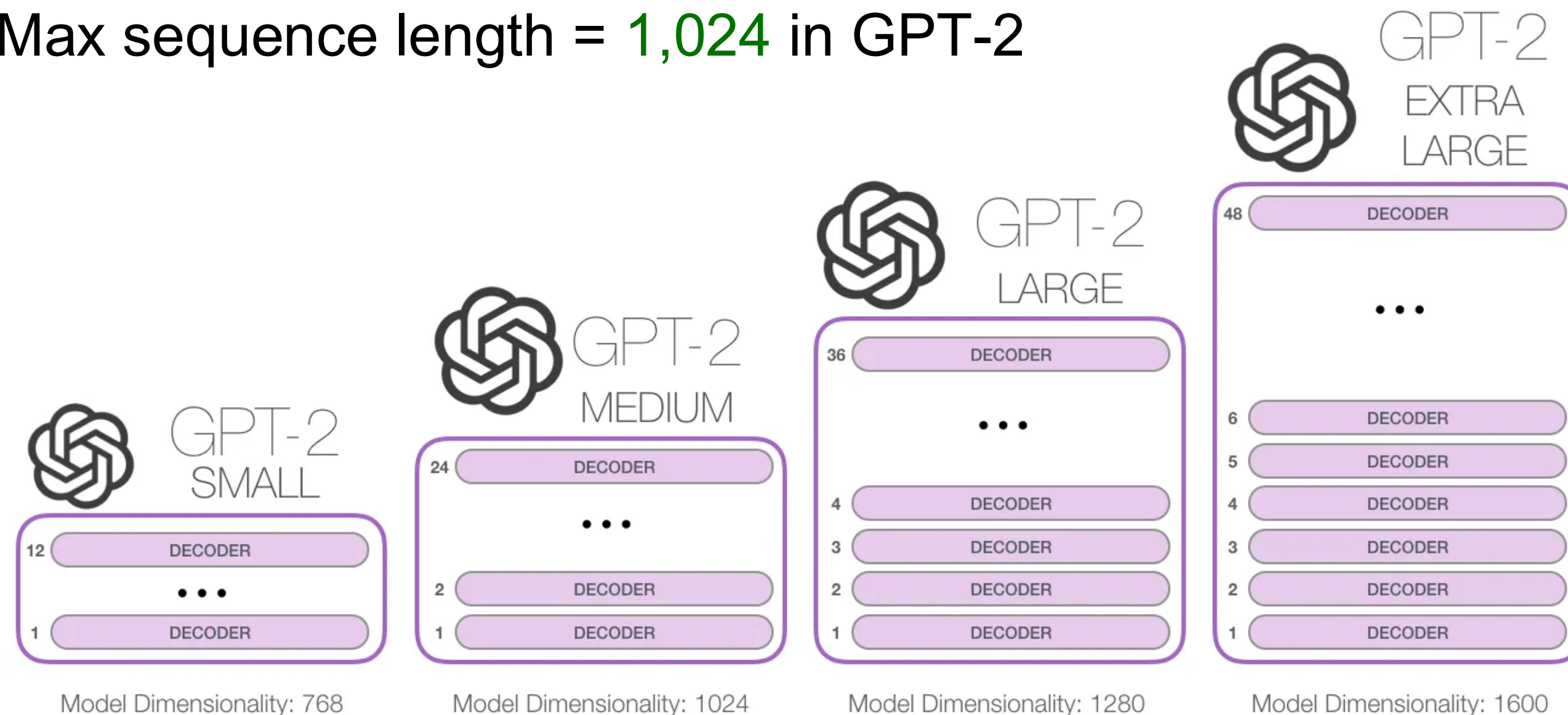
[Zhou et al. 2020]: A Transformer-based compiler model (GO-one) speeds up a Transformer model!

Model (#devices)	GO-one (s)	HP (s)	METIS (s)	HDP (s)	Run time speed up over HP / HDP	Search speed up over HDP
2-layer RNNLM (2)	0.173	0.192	0.355	0.191	9.9% / 9.4%	2.95x
4-layer RNNLM (4)	0.210	0.239	0.503	0.251	13.8% / 16.3%	1.76x
8-layer RNNLM (8)	0.320	0.332	OOM	0.764	3.8% / 58.1%	27.8x
2-layer GNNMT (2)	0.301	0.384	0.344	0.327	27.6% / 14.3%	30x
4-layer GNNMT (4)	0.350	0.469	0.466	0.432	34% / 23.4%	58.8x
8-layer GNNMT (8)	0.440	0.562	OOM	0.693	21.7% / 36.5%	7.35x
2-layer Transformer-XL (2)	0.223	0.268	0.37	0.262	20.1% / 17.4%	40x
4-layer Transformer-XL (4)	0.230	0.27	OOM	0.259	17.4% / 12.6%	26.7x
8-layer Transformer-XL (8)	0.350	0.46	OOM	0.425	23.9% / 16.7%	16.7x
Inception (2) b64	0.229	0.312	OOM	0.301	26.6% / 23.9%	13.5x
AmoebaNet (2) b64	0.423	0.731	OOM	0.498	42.1% / 29.3%	21.0x
AmoebaNet (4)	0.394	0.44	0.426	0.418	26.1% / 6.1%	58.8x
2-stack 18-layer WaveNet (2)	0.317	0.376	OOM	0.354	18.6% / 11.7%	6.67x
4-stack 36-layer WaveNet (4)	0.659	0.988	OOM	0.721	50% / 9.4%	20x
GEOMEAN	-	-	-	-	20.5% / 18.2%	15x

Quadratic computation as a function of sequence length

Need to compute n^2 pairs of scores (= dot product) $O(n^2 d)$

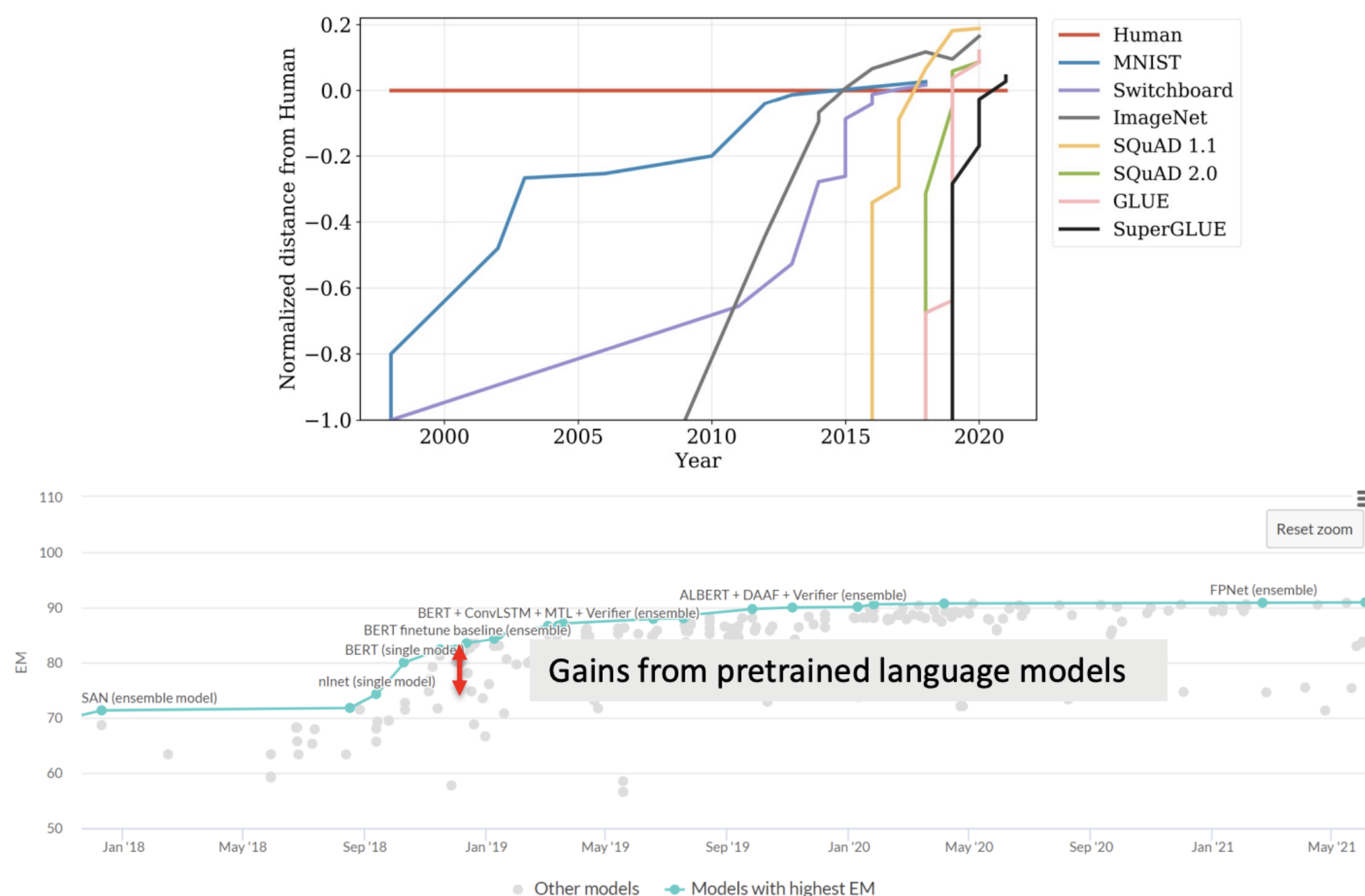
Max sequence length = 1,024 in GPT-2



What if we want to scale $n \geq 50,000$? For example, to work on long documents?

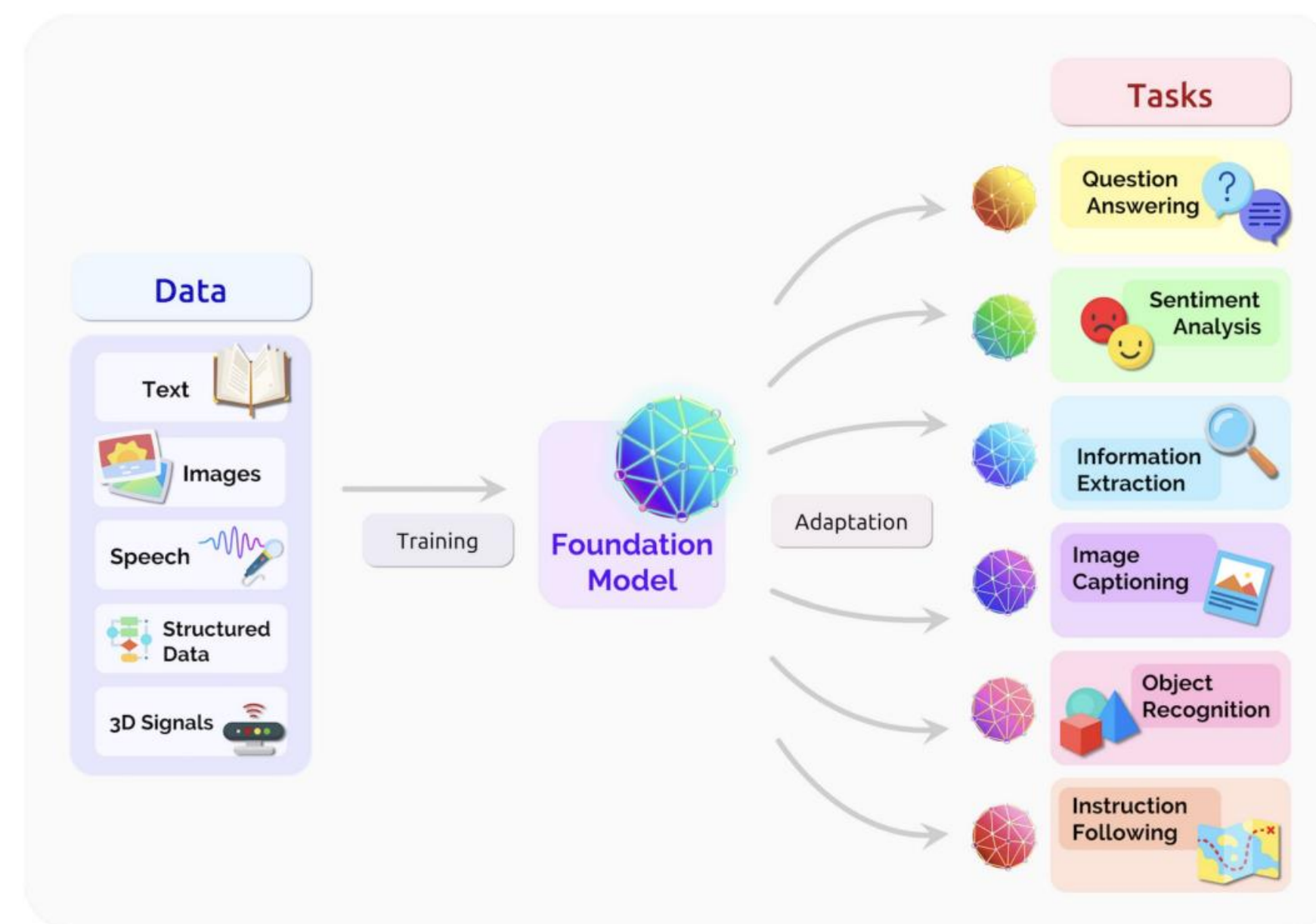
The Pre-training and Fine-tuning Paradigm

Pretraining Revolution



Pretraining has had a major, tangible impact on how well NLP systems work

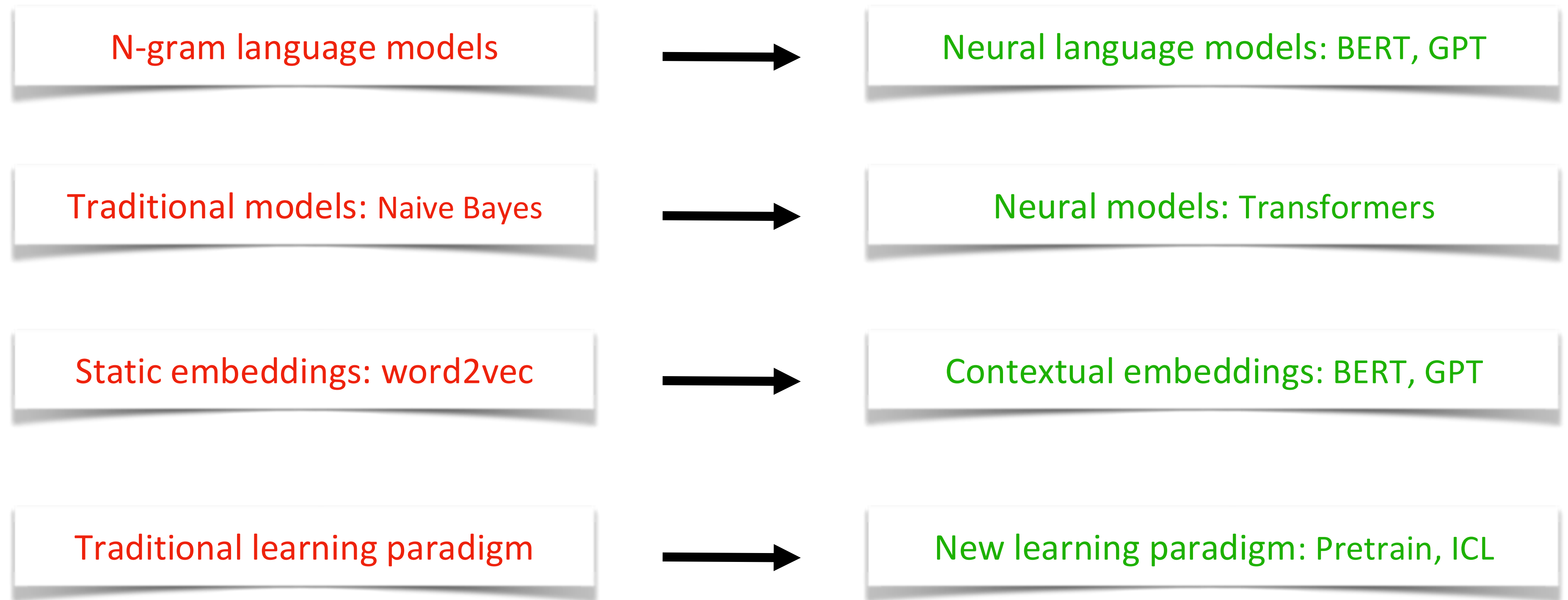
Pretraining - Scaling unsupervised learning on the Internet



Key ideas in pretraining

- Make sure your model can process large-scale, diverse datasets
- Don't use labeled data (otherwise you can't scale!)
- Compute-aware scaling

Traditional to modern NLP: training paradigm



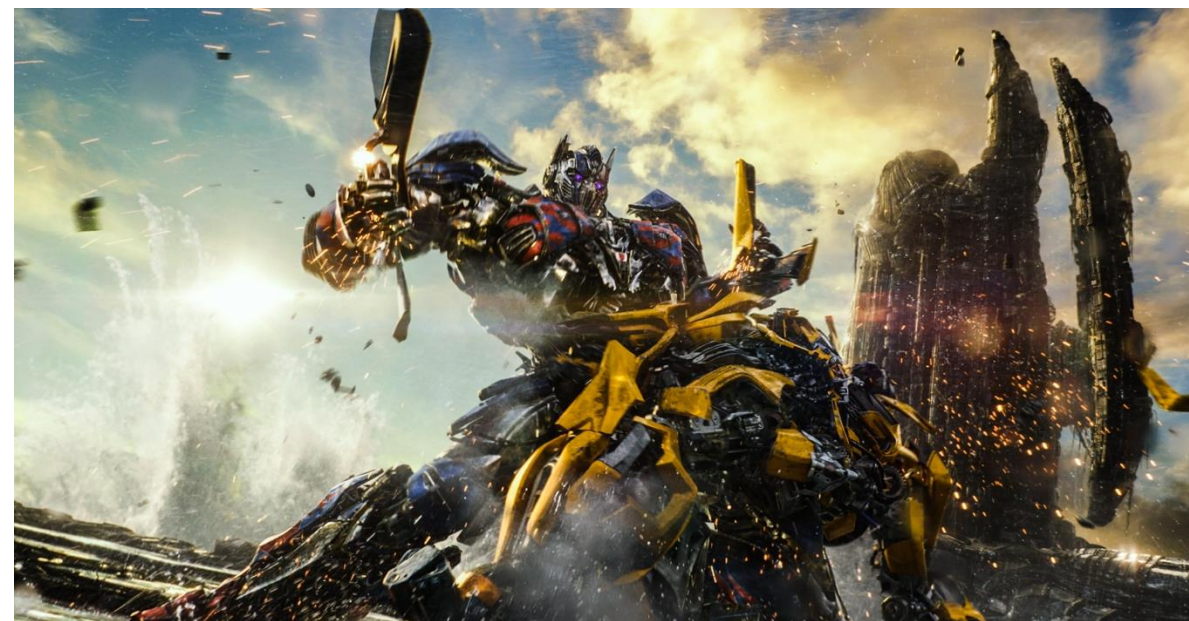
Question: How to train and use neural language models for different NLP tasks?

Example: Training Transformers for sentiment analysis

Task:



Model:



Transformers

Traditional learning paradigm

Supervised training/fine-tuning only, **NO pre-training**

- Collect (x, y) task training pairs

Data:

sentence	label
a stirring , funny and finally transporting re imagining of beauty and the beast and 1930s horror films	1
apparently reassembled from the cutting room floor of any given daytime soap	0
they presume their audience won't sit still for a sociology lesson	0
this is a visually stunning rumination on love , memory , history and the war between art and commerce	1
jonathan parker 's bartleby should have been the be all end all of the modern office anomie films	1

Traditional learning paradigm

Supervised training/fine-tuning only, **NO pre-training**

- Collect (x, y) task training pairs
- Randomly initialize your models $f(x)$ (e.g., vanilla Transformers)

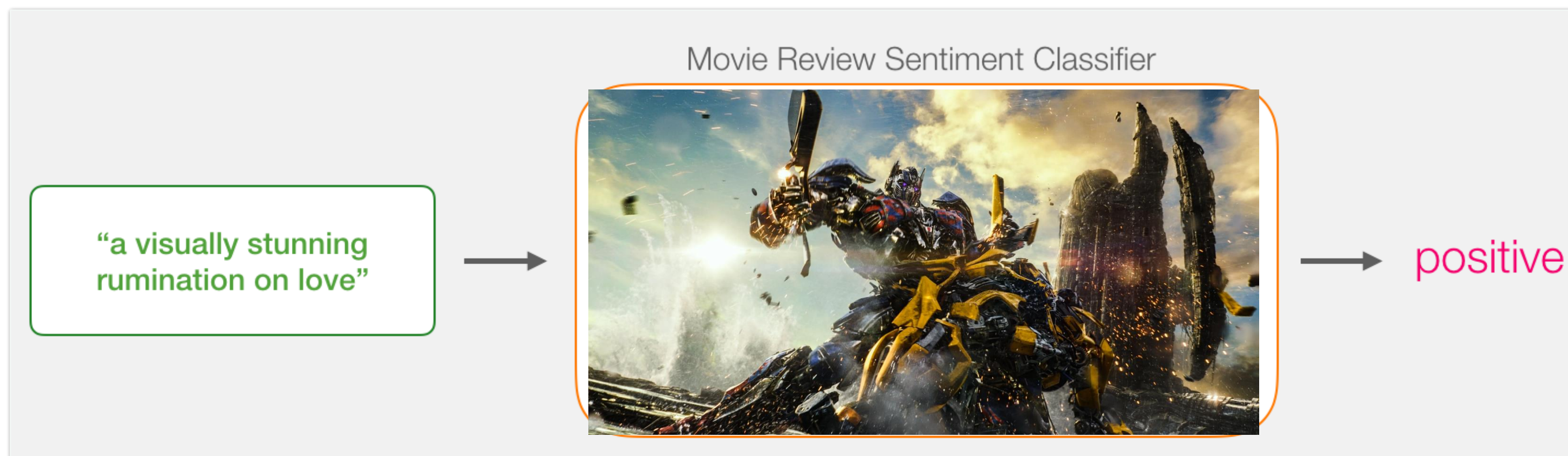


Randomly initialized Transformers
NO pretrained parameters used

Traditional learning paradigm

Supervised training/fine-tuning only, **NO pre-training**

- Collect (x, y) task training pairs
- Randomly initialize your models $f(x)$ (e.g., vanilla Transformers)
- Train $f(x)$ on (x, y) pairs

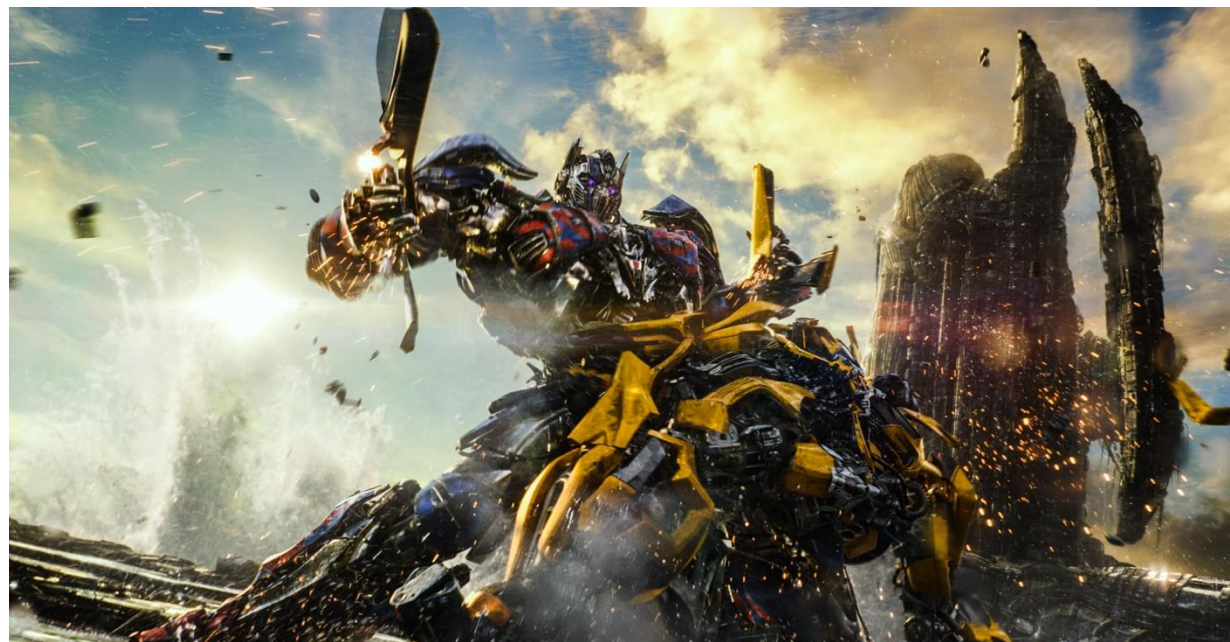


Train Transformers only on task labeled data₄₂

Traditional learning paradigm

Supervised training/fine-tuning only, **NO pre-training**

- Collect (x, y) task training pairs
- Randomly initialize your models $f(x)$ (e.g., vanilla Transformers)
- Train $f(x)$ on (x, y) pairs



Then you get a trained Transformers **ONLY** for sentiment analysis
The model can be: NB, LR, RNNs, LSTM too

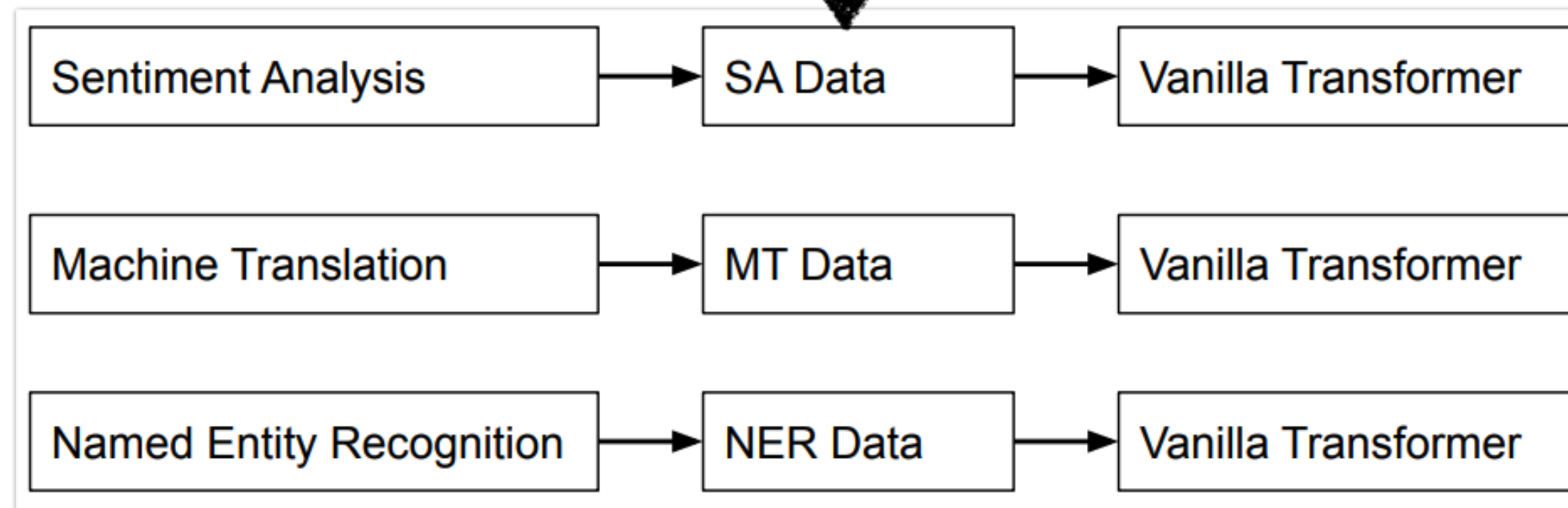
Traditional learning paradigm

Supervised training/fine-tuning only, **NO pre-training**

Train Transformer or other models separately for each task

sentence	label
a stirring , funny and finally transporting re imagining of beauty and the beast and 1930s horror films	1
apparently reassembled from the cutting room floor of any given daytime soap	0
they presume their audience won't sit still for a sociology lesson	0
this is a visually stunning rumination on love , memory , history and the war between art and commerce	1
jonathan parker 's bartleby should have been the be all end all of the modern office anomie films	1

Supervised fine-tuning only
NO pre-training



Modern learning paradigm

Pre-training + supervised training/fine-tuning

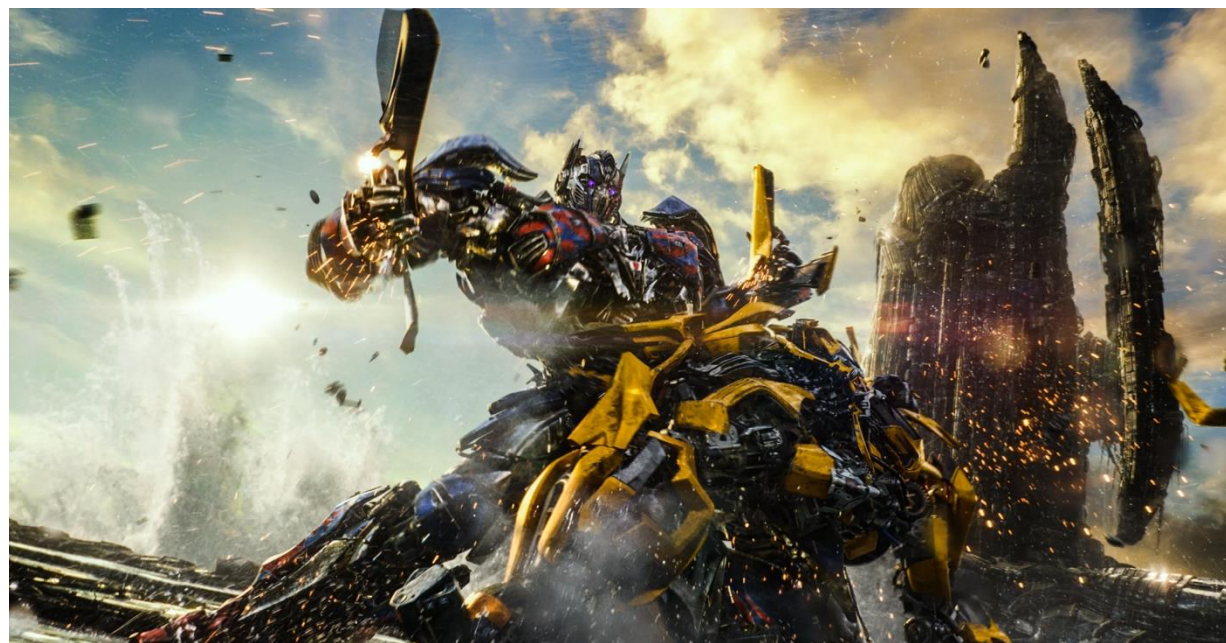
First train Transformer using a lot of general text using unsupervised learning. This is called **pretraining**.



Modern learning paradigm

Pre-training + supervised training/fine-tuning

First train Transformer using a lot of general text using unsupervised learning. This is called **pretraining**.



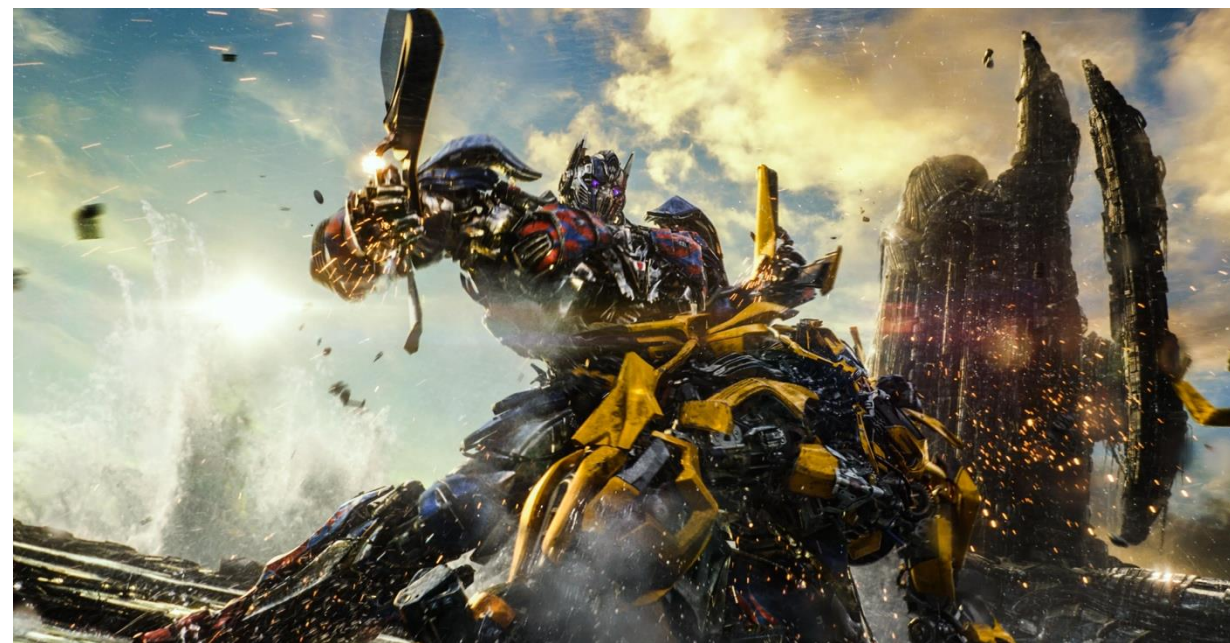
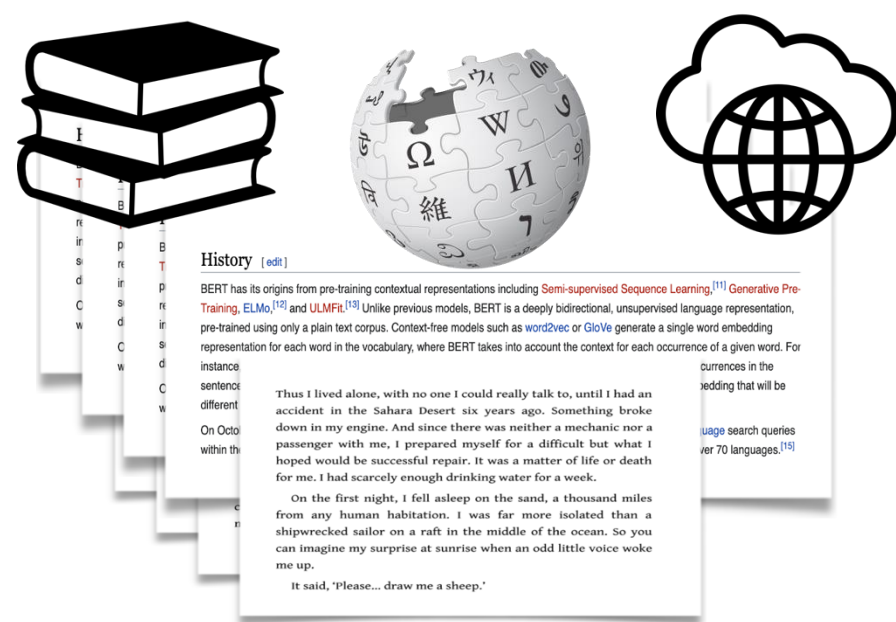
1. Randomly initialized Transformers



Modern learning paradigm

Pre-training + supervised training/fine-tuning

First train Transformer using a lot of general text using unsupervised learning. This is called **pretraining**.



**general
training objectives?**

2. Pre-train Transformers on large text data with **general objectives**



>2M GPU hours

Modern learning paradigm

Pre-training + supervised training/fine-tuning

First train Transformer using a lot of general text using unsupervised learning. This is called **pretraining**.

Then train the pretrained Transformer for a specific task using supervised learning. This is called **finetuning**.

Pretrained Transformers



Sentiment Analysis

Finetuning

Machine Translation

Finetuning

Named Entity Recognition

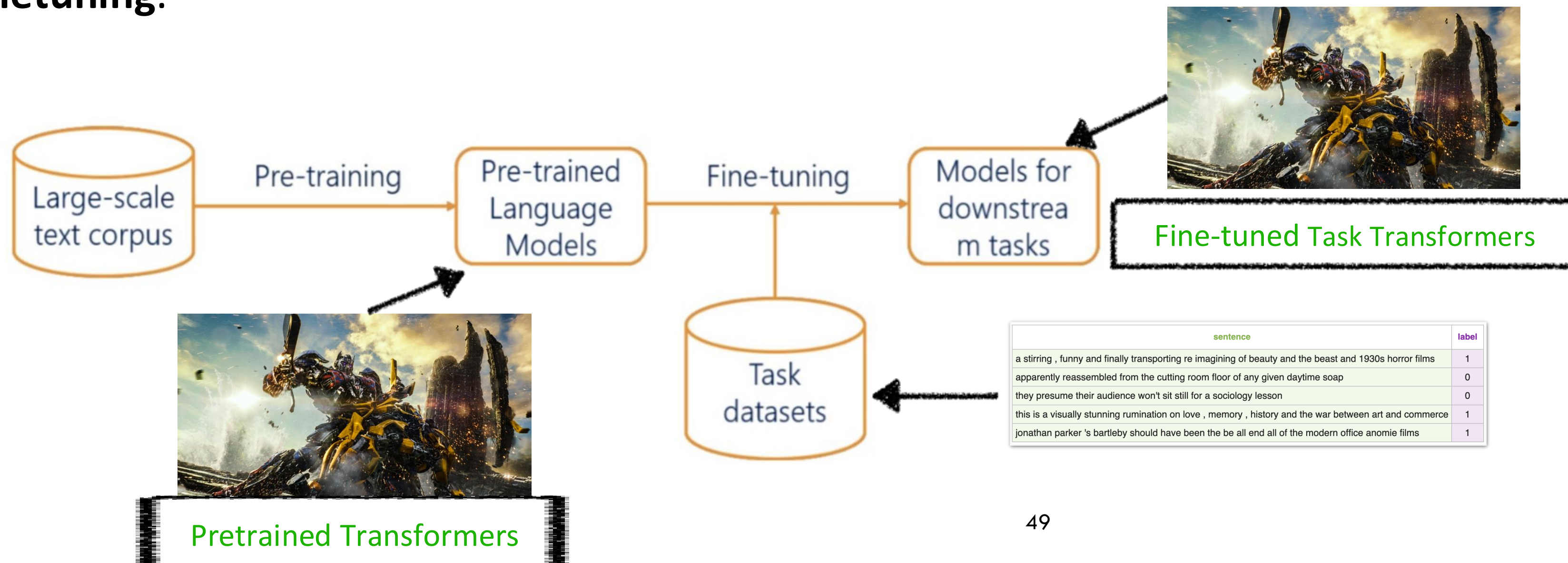
Finetuning

Modern learning paradigm

Pre-training + supervised training/fine-tuning

First train Transformer using a lot of general text using unsupervised learning. This is called **pretraining**.

Then train the pretrained Transformer for a specific task using supervised learning. This is called **finetuning**.



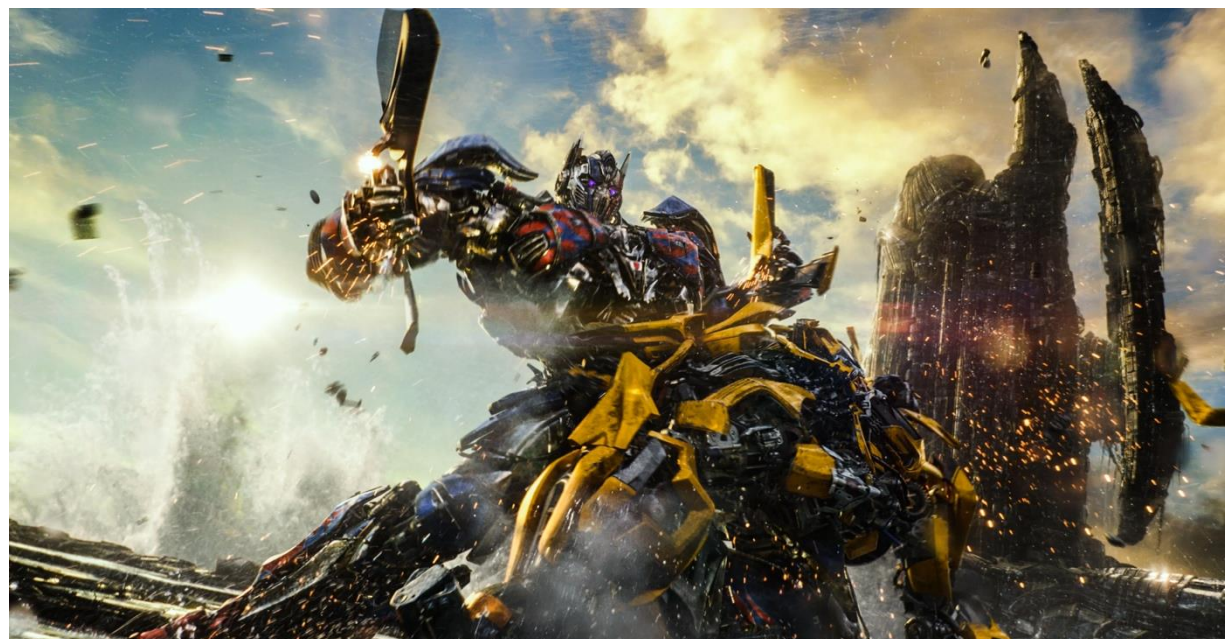
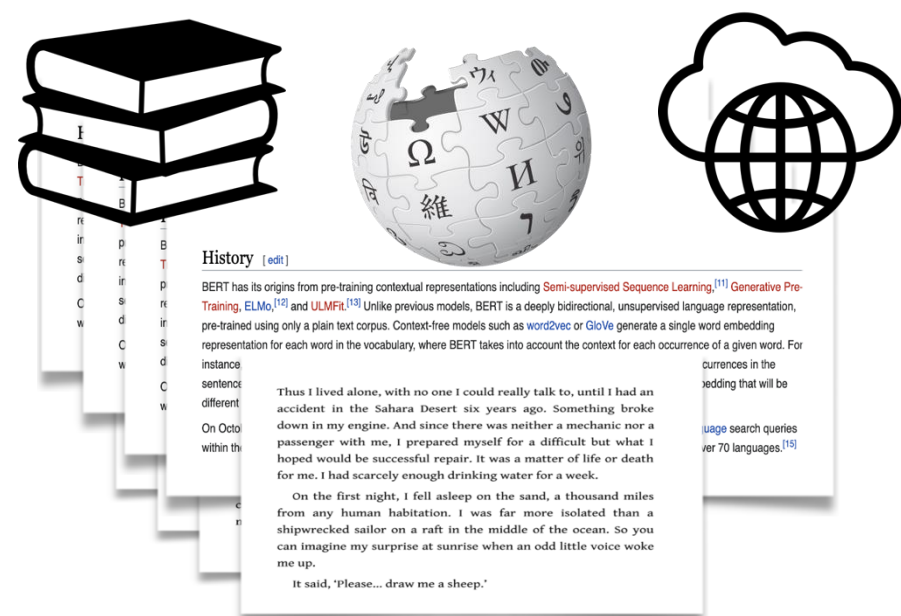
Advantages of pre-training

- **Leveraging rich underlying information** from abundant raw texts.
- **Reducing the reliance of task-specific labeled data** that is difficult or costly to obtain.
- **Initializing model parameters** for more **generalizable** NLP applications.
- **Saving training cost** by providing a reusable model checkpoints.
- **Providing robust representation** of language contexts.

Pretraining: training objectives?

During pretraining, we have a large text corpus (**no task labels**)

- **Key question: what labels or objectives used to train the vanilla Transformers?**

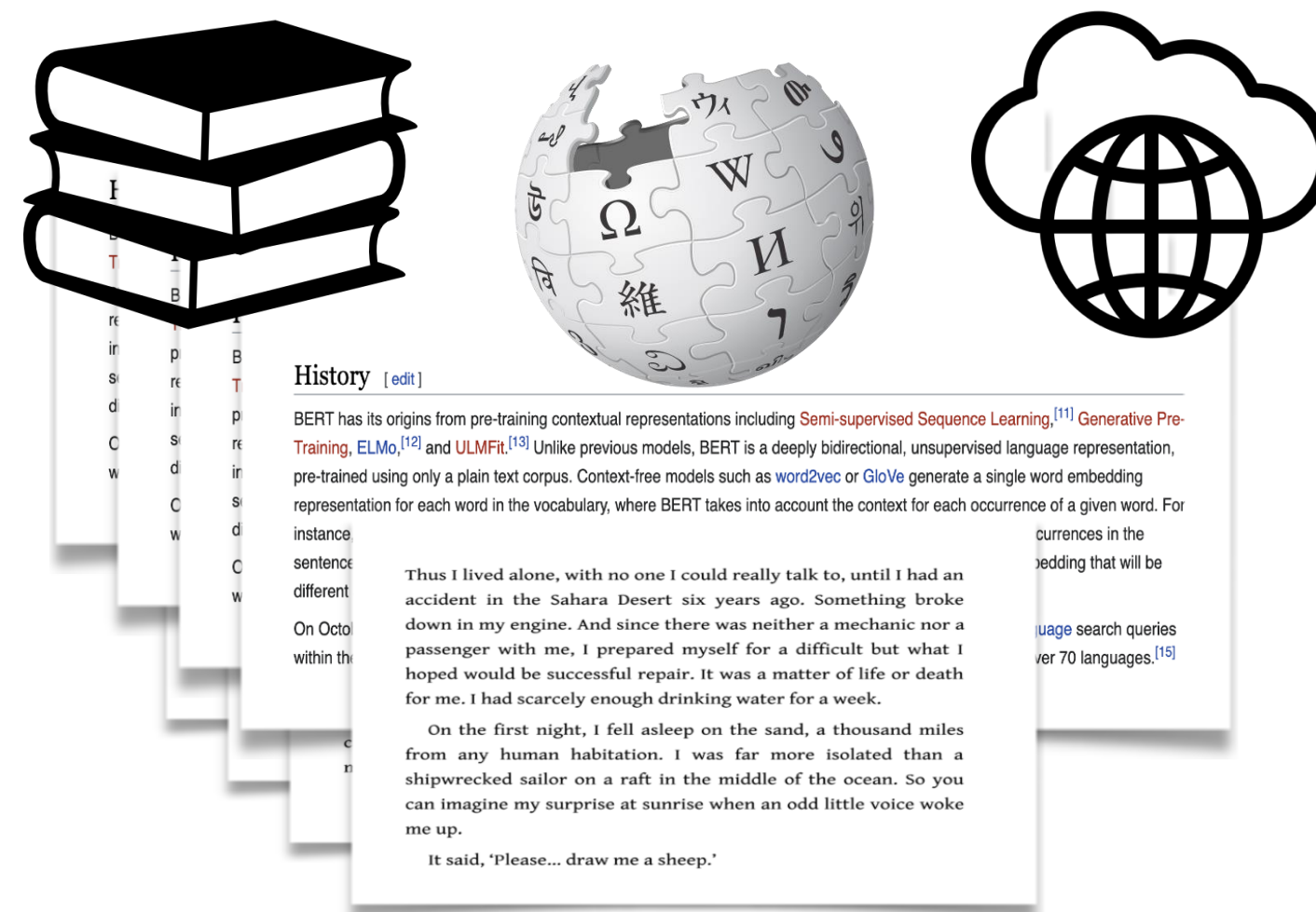


Training labels/objectives?

Pretraining: training objectives?

During pretraining, we have a large text corpus (**no task labels**)

- **Key question: what labels or objectives used to train the vanilla Transformers?**



Most of pre-training is reconstructing the input

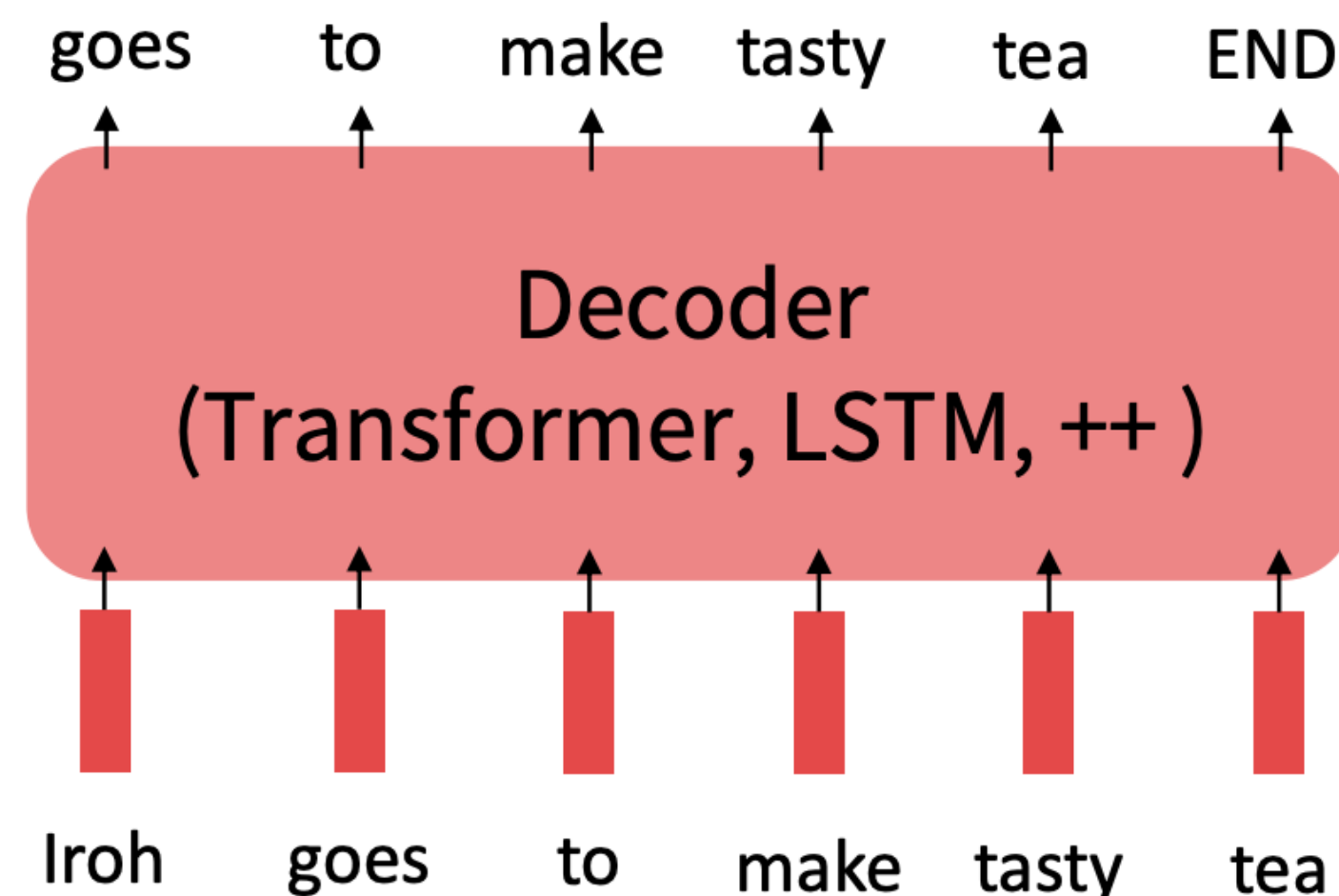
University of Southern California is located in ____ .

What can we learn from reconstructing the input?

- I went to the ocean and saw fish, turtles, _____ and seals.
 - General semantics
- I put ____ fork down on the table
 - Syntactic constraints
- The woman walked across the street checking for traffic over _____ shoulder.
 - Co-reference, relations between different entities within the sentence
- Overall, the value I got from the two hours watching it was the sum total of the popcorn and the drink. The movie was _____.
 - Sentiment

Pretraining: Language Models

- Recall the language modeling task:
 - Model $p_{\theta}(w_t | w_{1:t-1})$, the probability distribution over words given their past contexts.
 - There's lots of data for this! (In English.)
- Pretraining through language modeling:
 - Train a neural network to perform language modeling on a large amount of text.
 - Save the network parameters.



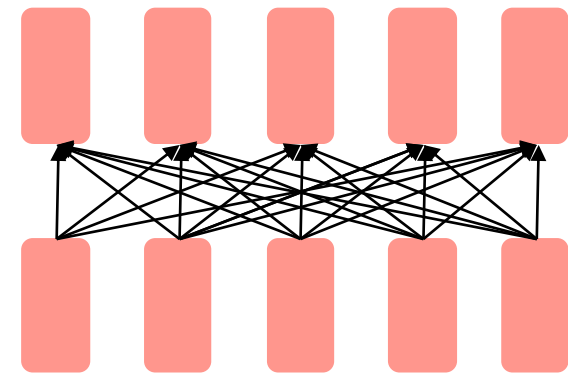
Semi-supervised Sequence Learning

Andrew M. Dai
Google Inc.
adai@google.com

Quoc V. Le
Google Inc.
qvl@google.com

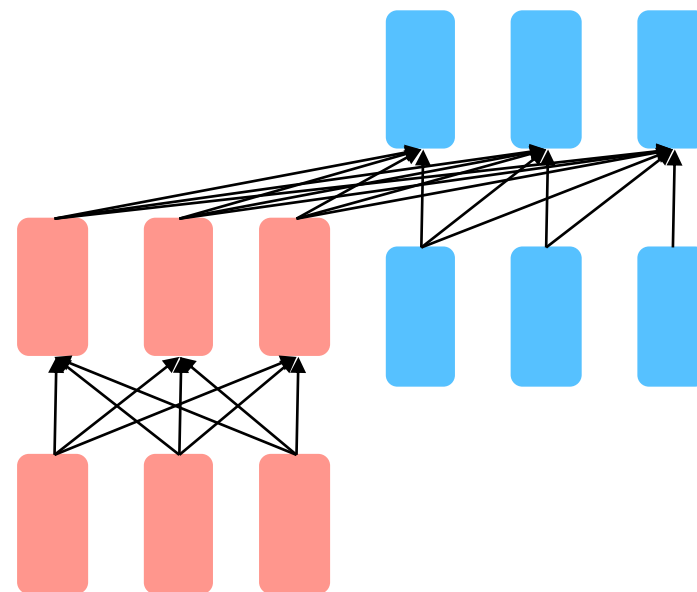
Pre-training architectures

Encoder



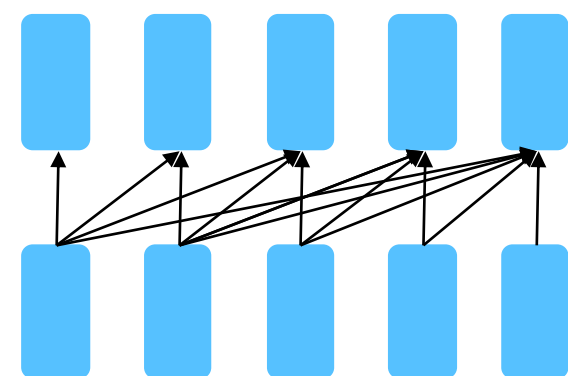
- Bidirectional; can condition on the future context

Encoder-Decoder



- Map two sequences of different length together

Decoder



- Language modeling; can only condition on the past context

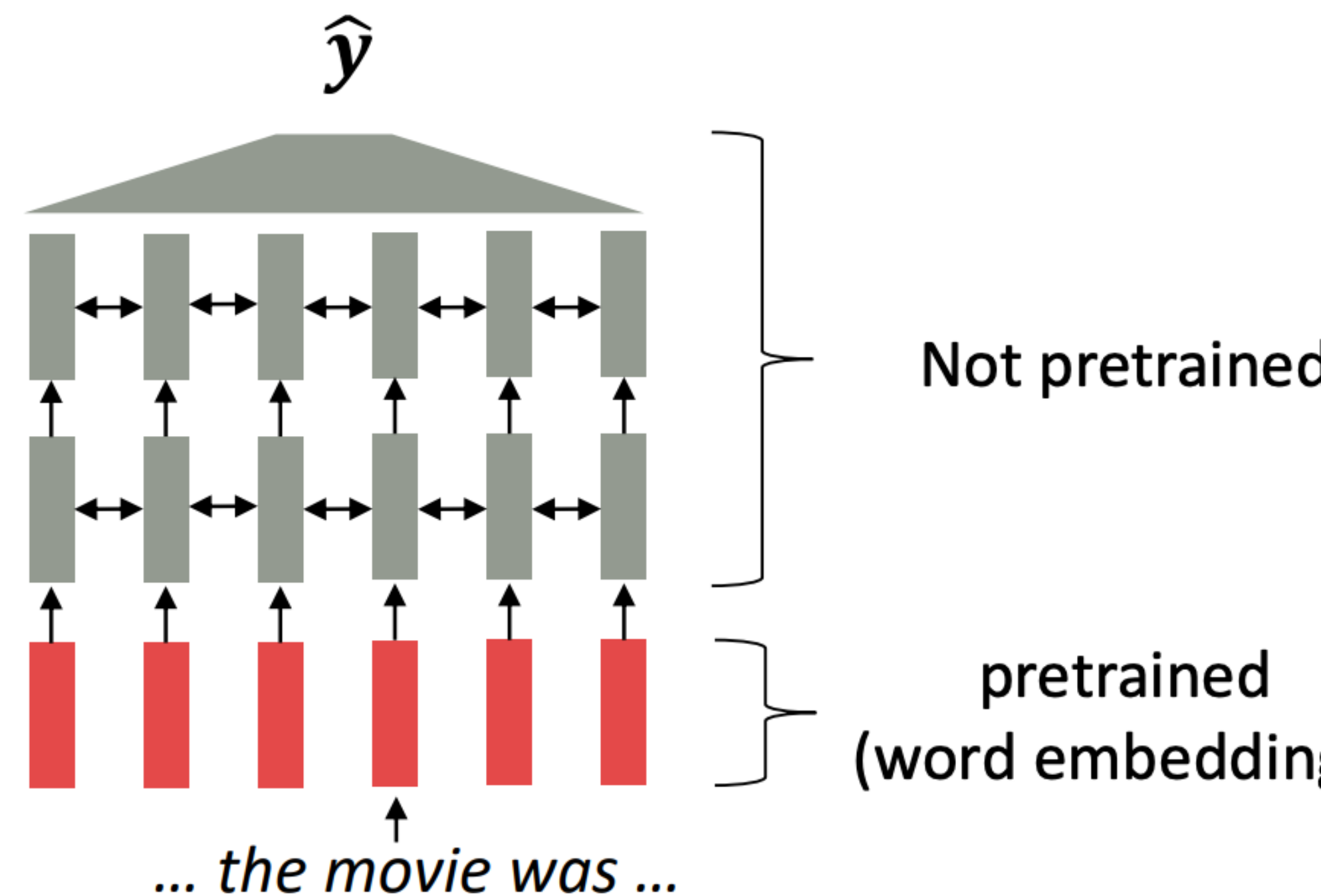
Word embeddings were pretrained too!

Previously (~2015):

- Start with pretrained word embeddings
 - Trained with limited context (windows)
- Learn how to incorporate context in an LSTM or Transformer while training on the task (e.g. sentiment classification)

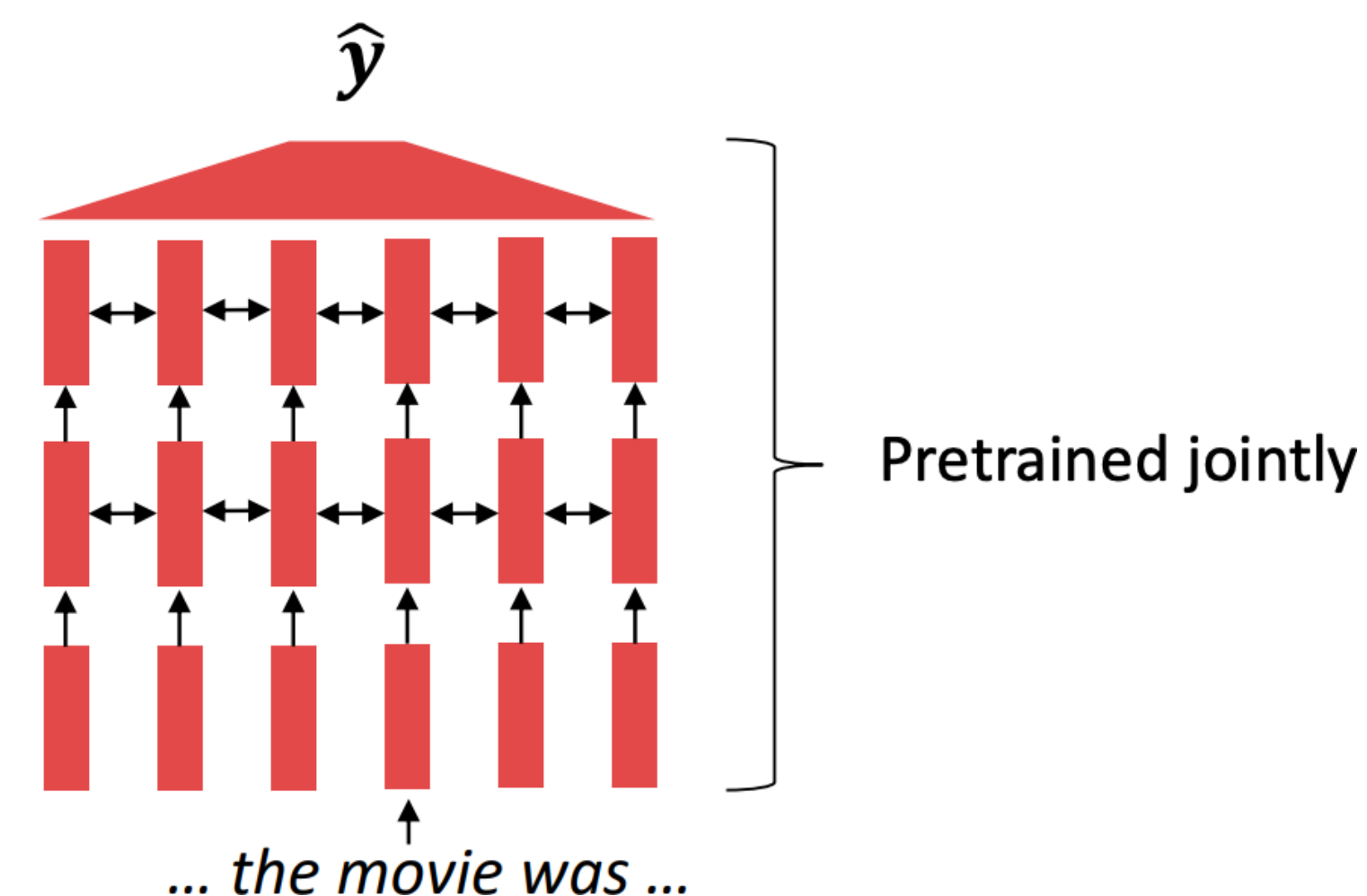
Some issues to think about:

- The training data we have for our downstream task (e.g. QA) must be sufficient to teach all contextual aspects of language
- Most of the parameters in our network are randomly initialized.



Pretraining Entire Models

- In modern NLP:
 - All (or almost all) parameters in NLP networks are initialized via pretraining.
 - This has been exceptionally effective at building strong:
 - representations of language
 - parameter initializations for strong NLP models.
 - probability distributions over language that we can sample from



[This model has learned how to represent entire sentences through pretraining]